



INDIVIDUAL ASSIGNMENT - LLSMS2012

Professor: LEONARDO IANIA

Author: JEAN DE BIOLLEY: 55612100

Date: May 2026

Contents

1	Time series	2
1.1	Question 1: AR(1) process	2
1.1.1	Sequence of conditional means	2
1.2	Sequence of conditional variances	3
1.3	Given ρ and σ_ε	3
1.3.1	i. Plot of the conditional mean	3
1.3.2	ii. Plot the conditional standard deviation bands	4
1.3.3	iii. Plot the prediction cones	4
1.3.4	iv. Simulate normal paths	5
1.4	Unconditional mean and variance	6
1.5	Question 2. Returns at different horizons	7
1.5.1	Mean log long-horizon return	7
1.5.2	Variance log long-horizon return	8
1.5.3	Standard deviation log long-horizon return	8
1.5.4	Annualizing monthly returns	9
1.5.5	Sharpe Ratio	9
1.5.6	The typical portfolio allocation ideal horizon	9
1.6	Fallacy of time-diversification	9
1.6.1	Effect of mean-reversion on long-horizon returns	10
1.6.2	Refresher on MA(1) processes	10
1.6.3	Mean and variance of long horizon returns of an MA(1) function	12
2	Stock Market	14
2.1	Efficient frontier	14
2.1.1	Part 1- Two risky assets and one risk-free asset	14
2.1.2	Impact of global events	14
2.1.3	Matrices of sample returns and variance-covariance	15
2.1.4	Portfolio optimization and the Efficient frontier	15
2.1.5	Global minimum variance portfolio (GMVP)	16
2.1.6	The tangency portfolio and the capital allocation line (CAL)	17
2.1.7	Plot of the efficient frontier	17
2.1.8	Part 2- Five risky assets and one risk-free asset	17
2.2	Stock returns and strategies	19
2.2.1	Winner and loser portfolios	22
2.2.2	Cumulative returns comparison	23

2.2.3	The 2009 crash	23
2.2.4	Similar cases	24
3	Bond Market	24
3.1	Test 1: Test of long yield changes over the life of the short bond	25
3.1.1	Model specification	25
3.1.2	Empirical results	26
3.1.3	Results analysis	27
3.2	Test 2: Test of short yield changes over the life of the long bond	27
3.2.1	Model Specification	27
3.2.2	Empirical results	28
3.2.3	Results analysis	29
3.3	Conclusion of both tests	29
	References	29
A	Source Code	29
A.1	Python Script for question 1: Standard deviation bands, prediction cones and simulated paths	29
A.2	Python script: plots for the IRF on a MA(1) process	32
A.3	Python script for the efficient frontier and capital allocation line: 2 assets	33
A.4	Python script for efficient frontier and CAL: 5 assets	35
A.5	R scripts for question 3.2: Industry momentum	38
A.6	R script for question 4: Expectation hypothesis	45

1. Time series

1.1. Question 1: AR(1) process

A stock price x_t follows an AR(1) process, i.e.

$$x_t = \rho x_{t-1} + \epsilon_t$$

where $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$.

1.1.1 Sequence of conditional means

We have to find the sequence of conditional means starting at x_{t+1} .

$$x_{t+1} = \rho x_t + \epsilon_{t+1}$$

We take the conditional mean:

$$\mathbf{E}_t(x_{t+1} \mid \mathbf{I}_t) = \mathbf{E}_t(\rho x_t) + \mathbf{E}_t(\epsilon_{t+1})$$

with $\mathbf{E}_t(\epsilon_{t+1}) = 0$. Thus,

$$\mathbf{E}_t(x_{t+1} \mid \mathbf{I}_t) = \mathbf{E}_t(\rho x_t) = \rho x_t$$

For $t = t + 2$:

$$\mathbf{E}_t(x_{t+2} \mid \mathbf{I}_{t+1}) = \mathbf{E}_{t+1}(\rho x_{t+1}) + \mathbf{E}_{t+1}(\epsilon_{t+2}) = \rho(\rho x_t) = \rho^2 x_t$$

For $t = t + j$:

$$\mathbf{E}_t(x_{t+j} \mid \mathbf{I}_{t+j-1}) = \rho^j x_t$$

1.2. Sequence of conditional variances

We want to find the sequence of conditional variances $\mathbf{Var}_t(x_{t+1}), \mathbf{Var}_t(x_{t+2}), \dots, \mathbf{Var}_t(x_{t+j})$. We begin with a AR(1) process, i.e.

$$x_t = \rho x_{t-1} + \epsilon_t$$

where $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$

We begin with the first iteration:

$$x_{t+1} = \rho x_t + \epsilon_{t+1}$$

$$\mathbf{Var}_t(x_{t+1}) = \mathbf{Var}_t(\rho x_t + \epsilon_{t+1}),$$

$$\mathbf{Var}_t(\rho x_t) = 0 \text{ at time } t.$$

$$\mathbf{Var}_t(x_{t+1}) = \mathbf{Var}_t(\epsilon_{t+1}) = \sigma_\epsilon^2$$

At the second iteration:

$$x_{t+2} = \rho x_{t+1} + \epsilon_{t+2} = \rho(\rho x_t + \epsilon_{t+1}) + \epsilon_{t+2}$$

$$= \rho^2 x_t + \rho \epsilon_{t+1} + \epsilon_{t+2}$$

$$\mathbf{Var}_t(x_{t+2}) = \mathbf{Var}_t(\rho^2 x_t + \rho \epsilon_{t+1} + \epsilon_{t+2}) = \rho^2 \sigma_\epsilon^2 + \sigma_\epsilon^2$$

At the third iteration:

$$x_{t+3} = \rho(\rho^2 x_t + \rho \epsilon_{t+1} + \epsilon_{t+2}) + \epsilon_{t+3} = \rho^3 x_t + \rho^2 \epsilon_{t+1} + \rho \epsilon_{t+2} + \epsilon_{t+3}$$

$$\mathbf{Var}_t(x_{t+3}) = \mathbf{Var}_t(\rho^3 x_t + \rho^2 \epsilon_{t+1} + \rho \epsilon_{t+2} + \epsilon_{t+3})$$

$$= \rho^4 \sigma_\epsilon^2 + \rho^2 \sigma_\epsilon^2 + \sigma_\epsilon^2 = \sigma_\epsilon^2(1 + \rho^2 + \rho^4)$$

We follow the pattern for the jth step:

$$\mathbf{Var}_t(x_{t+j}) = \sigma_\epsilon^2(1 + \rho^2 + \dots + \rho^{2j-2})$$

Using the formula for the limit of the sum of a finite geometric series ($S_n = \frac{1-r^{n+1}}{1-r}$)

We obtain :

$$\mathbf{Var}_t(x_{t+j}) = \sigma_\epsilon^2 \frac{1 - (\rho^2)^j}{1 - \rho^2} = \sigma_\epsilon^2 \frac{1 - \rho^{2j}}{1 - \rho^2}$$

We can note that in case of stationarity, i.e. $|\rho| < 1$, the term ρ^{2j} disappears.

1.3. Given ρ and σ_ϵ

We suppose $\rho = 0.95, \sigma_\epsilon = 0.25, x_t = 0 \quad \forall t \leq 0$, and at time 1 we see $x_1 = 1$.

1.3.1 i. Plot of the conditional mean

We know the process is stationary as $|\rho| < 1$. This means that when we keep increasing the exponential multiplier of ρ , we inevitably go back to 0, the long-term mean. Mathematically, the expectation of x_{t+j} is $(\mathbf{E}_t(x_{t+j})) = \rho^{t+j-1} x_t$.

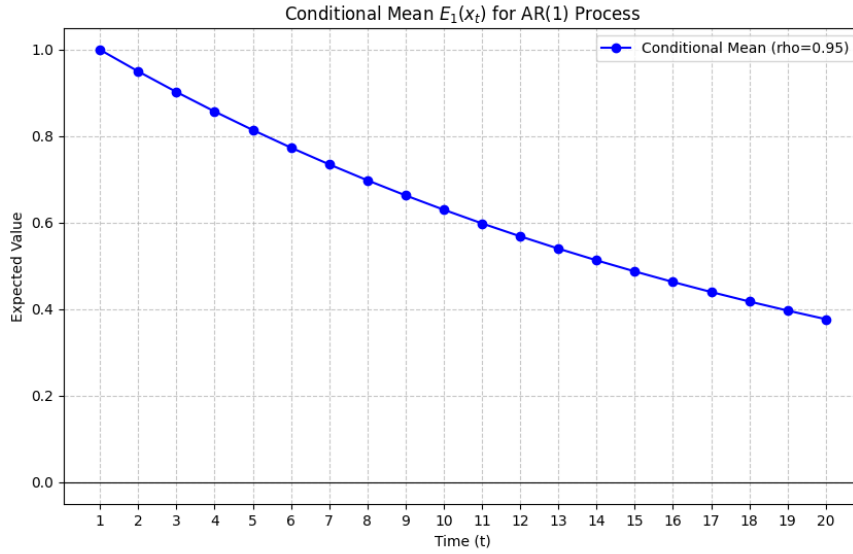


Figure 1: The plot illustrates the expected values of the AR(1) process slowly descending to 0 at a 5% decline rate, given an initial observation of $x_1 = 1$

1.3.2 ii. Plot the conditional standard deviation bands

We compute the plus and minus one conditional standard deviation bands. The bands are calculated as $E_1(x_t) \pm \sqrt{Var_1(x_t)}$.

We take the conditional variance from 1.2.b and conditional mean from 1.3.i. At time $t = 1$, we get the following conditional mean and variance:

$$E_1(x_t) = \rho^{t-1}x_1$$

$$Var_1(x_t) = \sigma_\epsilon^2 \frac{1 - \rho^{2(t-1)}}{1 - \rho^2}$$

1.3.3 iii. Plot the prediction cones

The Chebyshev Inequality tells us that a minimum proportion of data will fall within a certain number of standard deviations from the mean, regardless of the shape of the distribution.

We will compare this method of quantifying uncertainty to the one we used above.

For a random variable X with mean μ and standard deviation σ , and for any real number $k > 1$:

$$P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$$

To ensure at least $(1 - \alpha)$ probability stays inside the interval, we set $\frac{1}{k^2} = \alpha \implies k = \frac{1}{\sqrt{\alpha}}$.

- For 90% ($\alpha = 0.10$): $k = \frac{1}{\sqrt{0.1}} \approx 3.162$
- For 95% ($\alpha = 0.05$): $k = \frac{1}{\sqrt{0.05}} \approx 4.472$

The result is clear, Chebyshev intervals are much wider than the normal prediction cones. They are more cautious as they can't rely on a specific distribution.

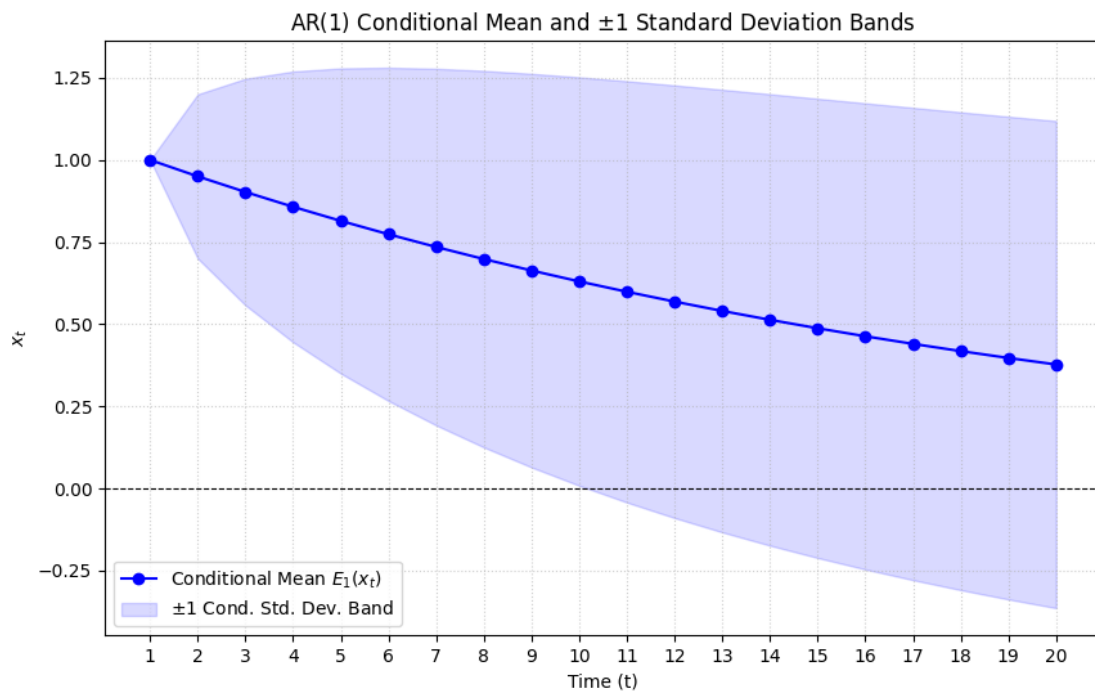


Figure 2: The plot illustrates the expected values of the AR(1) process with 1-standard deviation confidence bands

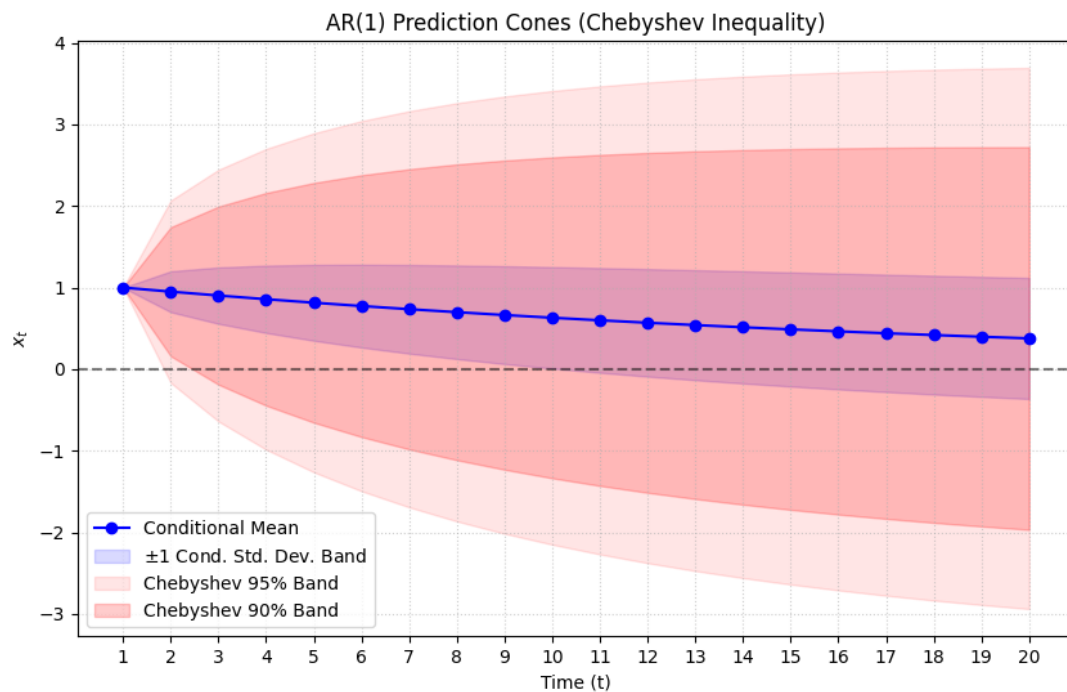


Figure 3: Chebyshev and normal intervals around an AR(1) process.

1.3.4 iv. Simulate normal paths

To create all the random paths, I used the python function `np.random.normal`

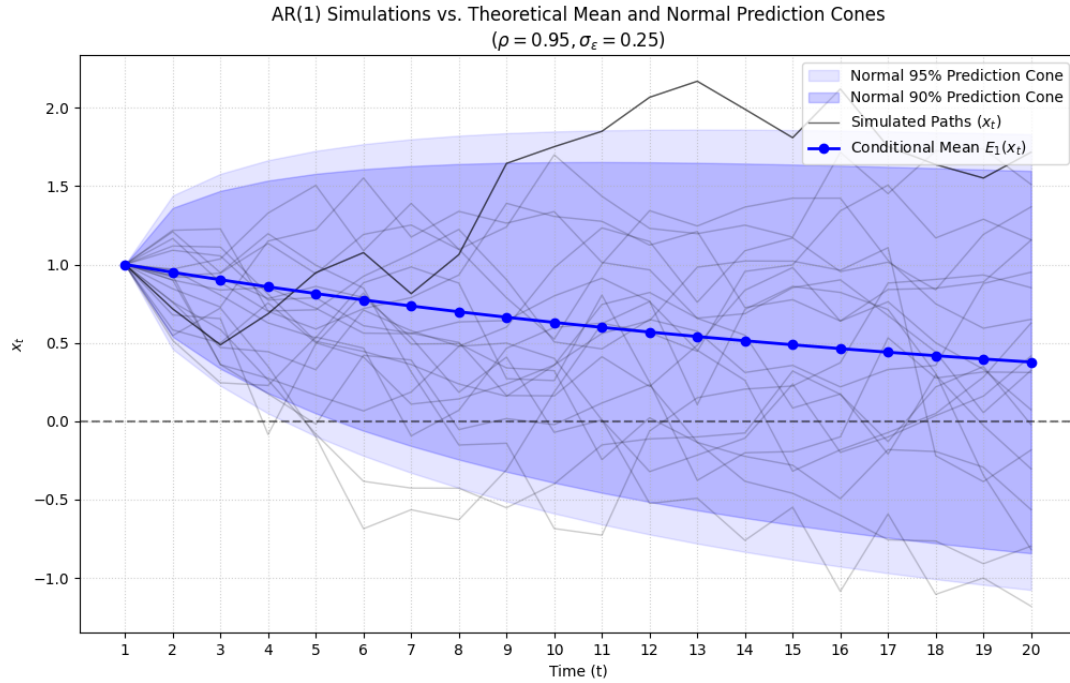


Figure 4: The figure illustrates 20 normal randomly simulated paths for an AR(1) process with $\rho = 0.95$ and $\sigma = 0.25$. The blue cones are the 90% and 95% confidence bands around the process.

Most path stay within the confidence bands.

1.4. Unconditional mean and variance

To find the unconditional mean and variance of an AR(1) process, we assume the process is stationary ($|\rho| < 1$) and that the mean and variance are constant.

We first find the **unconditional mean**. We start with the AR(1) process equation: $x_t = \rho x_{t-1} + \epsilon_t$

$$E(x_t) = E(\rho x_{t-1} + \epsilon_t)$$

Because $E(\epsilon_t) = 0$:

$$E(x_t) = \rho E(x_{t-1}) + E(\epsilon_t)$$

$$E(x_t) = \rho E(x_{t-1})$$

Because the process is stationary, the unconditional mean is constant for all t

$$E(x_t) = E(x_{t-1}) = \mu$$

Thus

$$\mu = \rho \mu \implies \mu(1 - \rho) = 0$$

Because of stationarity, $\rho \neq 1$. Thus, the only solution is:

$$E(x_t) = 0$$

To find the **unconditional variance**, we begin with the formula found in question 1.2.

$$Var_t(x_{t+j}) = \sigma_\epsilon^2 \frac{1 - \rho^{2j}}{1 - \rho^2}$$

In a stationary process, we can generalize the formula by removing the ρ^{2j} term. We get:

$$Var_t(x_{t+j}) = \frac{\sigma_\epsilon^2}{1 - \rho^2}$$

1.5. Question 2. Returns at different horizons

1.5.1 Mean log long-horizon return

We have a sequence of one-period log returns $\{r_t, r_{t+1}, \dots, r_{t+k}\}$. These returns are independent and identically distributed with a constant mean and variance. The k -period mean log long-horizon return is the expectation of the sum of the individual one-period log returns over that horizon:

$$E(r_{t+1} + r_{t+2} + \dots + r_{t+k})$$

We generalize the formula and are able to say that the expectation of a sum is the sum of the expectations because the returns are i.i.d.:

$$E(r_{t+1} + r_{t+2} + \dots + r_{t+k}) = E\left(\sum_{j=1}^k r_{t+j}\right) = \sum_{j=1}^k (E(r_{t+j}))$$

$$E(r_{t+j}) = \mu, \forall t, j$$

Thus:

$$\sum_{j=1}^k (E(r_{t+j})) = k\mu$$

The mean log long-horizon return is a linear function of the horizon k and the one-period mean μ .

For the mean annualized log long horizon, we follow the same process with a factor of $\frac{1}{k}$.

$$\mathbb{E}\left[\frac{1}{k} \left(\sum_{j=1}^k r_{t+j}\right)\right] = \frac{1}{k} \mathbb{E}\left[\left(\sum_{j=1}^k r_{t+j}\right)\right] = \frac{1}{k} (k\mu) = \mu$$

Assuming a mean annual return of 6%, the mean monthly return is 0.5%:

$$\mu_m = \frac{\mathbb{E}[r_a]}{12} = \frac{0.06}{12} = 0.005$$

$$\text{Annualized Mean} = \mu_m \times 12 = 0.005 \times 12 = 0.06$$

This is not surprising, and proves that the mean scales linearly in time.

1.5.2 Variance log long-horizon return

We want to find the variance log long-horizon return $\mathbf{VAR}(r_{t+1} + r_{t+2} + \dots + r_{t+k})$ and of the annualized log long horizon $\mathbf{VAR}[\frac{1}{k}(r_{t+1} + r_{t+2} + \dots + r_{t+k})]$ return as a function of the horizon k and the one-period variance σ^2 .

In the last question, we found that:

$$E(r_{t+1} + r_{t+2} + \dots + r_{t+k}) = E\left(\sum_{j=1}^k r_{t+j}\right) = \sum_{j=1}^k (E(r_{t+j}))$$

Now we do the same with the variance:

$$\text{Var}\left(\sum_{j=1}^k r_{t+j}\right) = \sum_{j=1}^k \text{Var}(r_{t+j})$$

And $\text{Var}(r_{t+j}) = \sigma^2 \quad \forall t, j$ Thus:

$$\text{Var}\left(\sum_{j=1}^k r_{t+j}\right) = k\sigma^2$$

We annualize the log long horizon variance of returns:

$$\text{Var}\left(\frac{1}{k} \sum_{j=1}^k r_{t+j}\right) = \frac{1}{k^2} \text{Var}\left(\sum_{j=1}^k r_{t+j}\right) k\sigma^2 = \frac{\sigma^2}{k}$$

This means that when k increases, the variance of the average return approaches zero. Combining the two findings, we understand that total variance increases with k . This makes sense intuitively because it's harder to predict events far in the future. And mean variance decreases when k increases. The individual noise of returns disappears, only leaving the LT trend.

1.5.3 Standard deviation log long-horizon return

We want to find the standard deviation log long-horizon return $\sqrt{\mathbf{VAR}(r_{t+1} + r_{t+2} + \dots + r_{t+k})}$ and of the annualized log long horizon $\sqrt{\mathbf{VAR}[\frac{1}{k}(r_{t+1} + r_{t+2} + \dots + r_{t+k})]}$ return as a function of the horizon k and the one-period standard deviation σ ; and find the standard deviation of monthly and daily returns when the sd of annual returns is 16%

Earlier, we found that:

$$\text{Var}\left(\sum_{j=1}^k r_{t+j}\right) = k\sigma^2$$

Thus:

$$\sqrt{\text{Var}\left(\sum_{j=1}^k r_{t+j}\right)} = \sqrt{k\sigma^2} = \sqrt{k}\sigma$$

For the annualized log long horizon returns, we find:

$$\sqrt{\text{Var}\left(\frac{1}{k} \sum_{j=1}^k r_{t+j}\right)} = \sqrt{\frac{1}{k^2} \text{Var}\left(\sum_{j=1}^k r_{t+j}\right)} = \frac{1}{k} \sqrt{k\sigma^2} = \frac{\sqrt{k}}{k} \sigma = \frac{\sigma}{\sqrt{k}}$$

When the sd of annual returns is 16%, the standard deviation of monthly returns is equal to $\frac{0.16}{\sqrt{12}} = 4.6\%$

For the number of days, I take 360, to use the ACT/360 convention. The standard deviation of daily returns is equal to $\frac{0.16}{\sqrt{360}} = 0.84\%$.

1.5.4 Annualizing monthly returns

A stock has a mean and standard deviation of monthly returns equal to 1%. We express these on an annualized basis using the formulas above.

$$\begin{aligned}\mu_a &= k\sigma = 12 \times 1\% = 12\% \\ \sigma_a &= \sqrt{k}\sigma = \sqrt{12} \times 1\% = 3.46\%\end{aligned}$$

1.5.5 Sharpe Ratio

Using the values of expectation and variance as functions of horizon k , we obtain:

$$\text{Sharpe ratio} = \frac{E\left(\sum_{j=1}^k r_{t+j}\right)}{\sqrt{\text{Var}\left(\sum_{j=1}^k r_{t+j}\right)}} = \frac{k\mu}{\sqrt{k}\sigma} = \sqrt{k}\mu \times \sigma^{-1}$$

The unit in which we quote the returns have an influence on sharpe ratio. With this formula, the number of observations increases the ratio by a factor of $\sqrt{k}$.

This means that for the same returns, an analyst can compute many Sharpe ratios for the same returns.

1.5.6 The typical portfolio allocation ideal horizon

The typical portfolio allocation formula says stock allocation should depend on the ratio of mean to variance of total return:

$$\text{stock share} = \frac{1}{\text{risk aversion}} \frac{\mathbf{E}(r)}{\mathbf{VAR}(r)}$$

Should an investor with a 10-year horizon allocate more to stocks than an investor with a 1-year horizon because “stocks are safer in the long run” and he can “wait out market declines?”.

Let’s look at the math shall we?

Taking $E(r) = k\mu$ and $\text{VAR}(r) = k\sigma^2$, we obtain:

$$\text{stock share} = \frac{1}{\text{risk aversion}} \frac{k\mu}{k\sigma^2} = \frac{1}{\text{risk aversion}} \frac{\mu}{\sigma^2}$$

As we can observe, this formula for portfolio allocation holds for any time horizon, as there is no variable k . With this formula alone, one can’t decide any market timing.

1.6. Fallacy of time-diversification

First, let’s dissect what the client told us. They said that long-horizon average returns are very stable, and the volatility of long-run average returns goes down with the square root of the horizon.

This is correct as we explained earlier. The mean annualized log long horizon is:

$$\mathbb{E}\left[\frac{1}{k}\left(\sum_{j=1}^k r_{t+j}\right)\right] = \mu$$

And the annualized log long horizon standard deviation of returns is:

$$\sqrt{\text{Var} \left(\frac{1}{k} \sum_{j=1}^k r_{t+j} \right)} = \sqrt{\frac{\sigma^2}{k}}$$

Averages are indeed very stable in the long run. The investor considers that the mean will stay constant at μ and volatility will decrease by a factor of $\sqrt{k}$.

However, what matters for investors are the total returns and not the averages. Total risk actually increases with time at a factor $\sqrt{k}$ as shown by the formula. Thus:

$$\text{Var} \left(\sum_{j=1}^k r_{t+j} \right) = k\sigma^2$$

$$\sqrt{\text{Var} \left(\sum_{j=1}^k r_{t+j} \right)} = \sqrt{k}\sigma$$

The longer someone invests, the wider the distribution of the final wealth becomes.

Let's take the example of a coin toss following a fair distribution. At each iteration, the probability of obtaining heads is $\frac{1}{2}$. An investor could then expect that after a large enough number of throws k , they will have obtained $\frac{k}{2}$ heads. However, because of the large number of throws and the independence of returns, the actual number of outcomes {head} can change widely when repeating the experience.

Stock market returns actually behave as random walks (autoregressive processes with a ρ coefficient equal to 1) and the increments of the random walk (returns) (r_t) are modeled as i.i.d. white noise.

The expectation of an increment of a martingale is the expectation of the previous step. If P_t is a RW, then $\mathbb{E}[P_{t+1}|P_t] = P_t$.

However, stocks don't usually land at the same values every time. The increments are i.i.d. and with constant mean, but they will systematically arrive at different levels as time passes, and the more time passes the less likely it is they go back to their original value.

Thus, risk systematically increases with k and the client was indeed engaged in a fallacy.

1.6.1 Effect of mean-reversion on long-horizon returns

What if returns have some "momentum" or "mean-reversion" in stock returns? Does this make "long horizon returns safer?"

To answer this question, we suppose returns follow the MA(1), Moving Average of order one, process:

$$r_t = \mu + \varepsilon_t + \delta\varepsilon_{t-1}$$

with ε_t independent over time and with $\mathbf{E}(\varepsilon_t) = 0$ and $\mathbf{VAR}(\varepsilon_t) = \sigma_\varepsilon^2$

1.6.2 Refresher on MA(1) processes

A. Moments of MA(1) compared to AR(1) :

Expected value:

$$E(r_t) = E(\mu) + \cancel{E(\varepsilon_t)} + \delta\cancel{\varepsilon_{t-1}} = \mu$$

Variance at first lag:

$$\begin{aligned} \text{VAR}(r_t) &= \cancel{\text{VAR}(\mu)} + \text{VAR}(\varepsilon_t + \delta\varepsilon_{t-1}) \\ &= \text{VAR}(\varepsilon_t) + \text{VAR}(\delta\varepsilon_{t-1}) = \sigma_\varepsilon^2 + \delta^2\sigma_\varepsilon^2 = \sigma_\varepsilon^2(1 + \delta^2) \end{aligned}$$

Coovariance at first lag:

We calculate $COV(r_t, r_{t-1})$ using the covariance formula:

$$COV(X, Y) = E[(X - E[X])(Y - E[Y])]$$

$$COV(r_t, r_{t-1}) = E[(r_t - E[r_t])(r_{t-1} - E[r_{t-1}])]$$

We can replace the expression $(r_t - E[r_t])$ with $\varepsilon_t + \delta\varepsilon_{t-1}$ because we find that:

$$(r_t - \mu) = \varepsilon_t + \delta\varepsilon_{t-1}$$

Thus:

$$COV(r_t, r_{t-1}) = E[(\varepsilon_t + \delta\varepsilon_{t-1})(\varepsilon_{t-1} + \delta\varepsilon_{t-2})]$$

$$\implies E[\varepsilon_t\varepsilon_{t-1} + \delta\varepsilon_t\varepsilon_{t-2} + \delta\varepsilon_{t-1}^2 + \delta^2\varepsilon_{t-1}\varepsilon_{t-2}]$$

$E[\varepsilon_i\varepsilon_j] = 0$ for $i \neq j$. The only non-zero term is $E[\delta\varepsilon_{t-1}^2]$

$$\implies COV(r_t, r_{t\pm 1}) = \delta\sigma_\varepsilon^2$$

Covariance at lag 2:

$$COV(r_t, r_{t-2}) = E[(\varepsilon_t + \delta\varepsilon_{t-1})(\varepsilon_{t-2} + \delta\varepsilon_{t-3})]$$

$$E[\varepsilon_t\varepsilon_{t-2} + \delta\varepsilon_t\varepsilon_{t-3} + \delta\varepsilon_{t-1}\varepsilon_{t-2} + \delta^2\varepsilon_{t-1}\varepsilon_{t-3}]$$

There are no overlapping indices for the error terms. Thus:

$$COV(r_t, r_{t\pm 2}) = 0$$

There are no overlapping indices from lag 2, so $\forall k \geq 2, COV(r_t, r_{t\pm k}) = 0$

It's easy to understand why we say this model has a "short-term" memory. After one lag, past shocks have no influence on the model. This differs completely from a stationary AR(1) model as the latter only removes the effect of a past shock at a rate of $(1 - \rho)$. This means that technically, shocks are never entirely meaningless even after many periods. The higher the factor ρ the less a model forgets past lags.

B. Impulse-response function (IRF) of r_t .

The IRF measures the impact of an additional shock of 1 on the function, assuming all other shocks are equal to 0.

Reminder, for our MA(1) process:

$$r_t = \mu + \varepsilon_t + \delta\varepsilon_{t-1}$$

We assume $\mu = 0$ for the purpose of looking at shocks. $\varepsilon_t = 1$ and $\varepsilon_{t\pm k} = 0 \quad \forall k \neq 1$

At time t:

$$r_t = 1 + \delta(0) = 1$$

At time t + 1:

$$r_t = \varepsilon_{t+1} + \delta\varepsilon_t = 0 + \delta \times 1 = \delta$$

At time t + 2:

$$r_t = 0 + \delta \times 0 = 0$$

If $\delta \geq 1$, the shock is amplified and a price of today leads to a greater price tomorrow, there is a "momentum of increase".

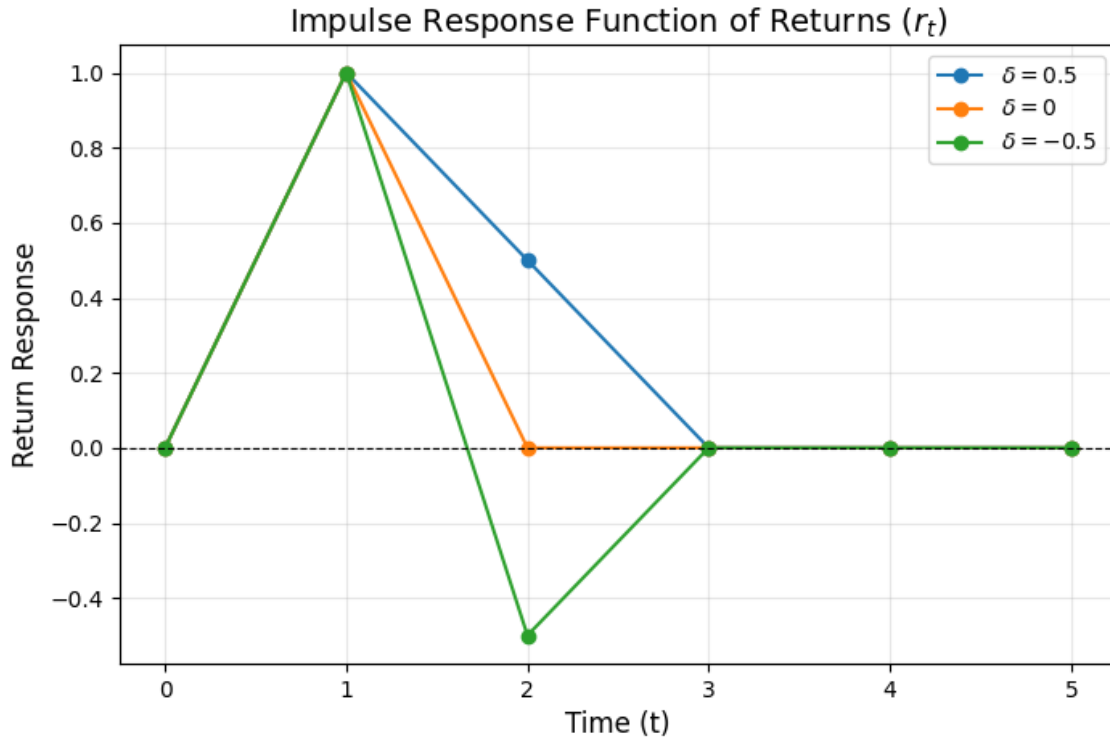


Figure 5: Impulse function of returns. A single shock is added at time $t = 1$. Depending on the δ value, IRF behaves differently

If $\delta = 0$, the price shifts to 1 and stays at that level forever. This is a random walk and returns are i.i.d.

If $\delta \leq 0$, the function is mean-reverting, the price gives back some of the increase and goes back in the direction of the original value.

We then find the IRF of the price, which are computed as the cumulative log returns using the `np.cumsum` function. We can also find the answers by hand. The price is defined as

$$p_t = \sum_{i=1}^t r_i$$

We suppose $r_1 = \varepsilon_1 = 1$ and all other $\varepsilon = 0$

At $t = 1$: $p_1 = r_1 = 1$

At $t = 2$: $p_2 = r_1 + r_2 = 1 + \delta$

At $t \geq 3$: $p_t = 1 + \delta + 0 + \dots = 1 + \delta$

1.6.3 Mean and variance of long horizon returns of an MA(1) function

As before, $r_{t+1} + r_{t+2} + \dots + r_{t+k} = \sum_{j=1}^k r_{t+j}$.

The expectation of a sum is the sum of expectations. As shown earlier and because $E(r_t) = E(r_{t+k}) \quad \forall k$, we obtain:

$$E \left[\sum_{j=1}^k r_{t+j} \right] = k\mu$$

We now will compute $VAR \left(\sum_{j=1}^k r_{t+j} \right)$. It will be more complicated as before as returns are now correlated, which wasn't the case earlier.

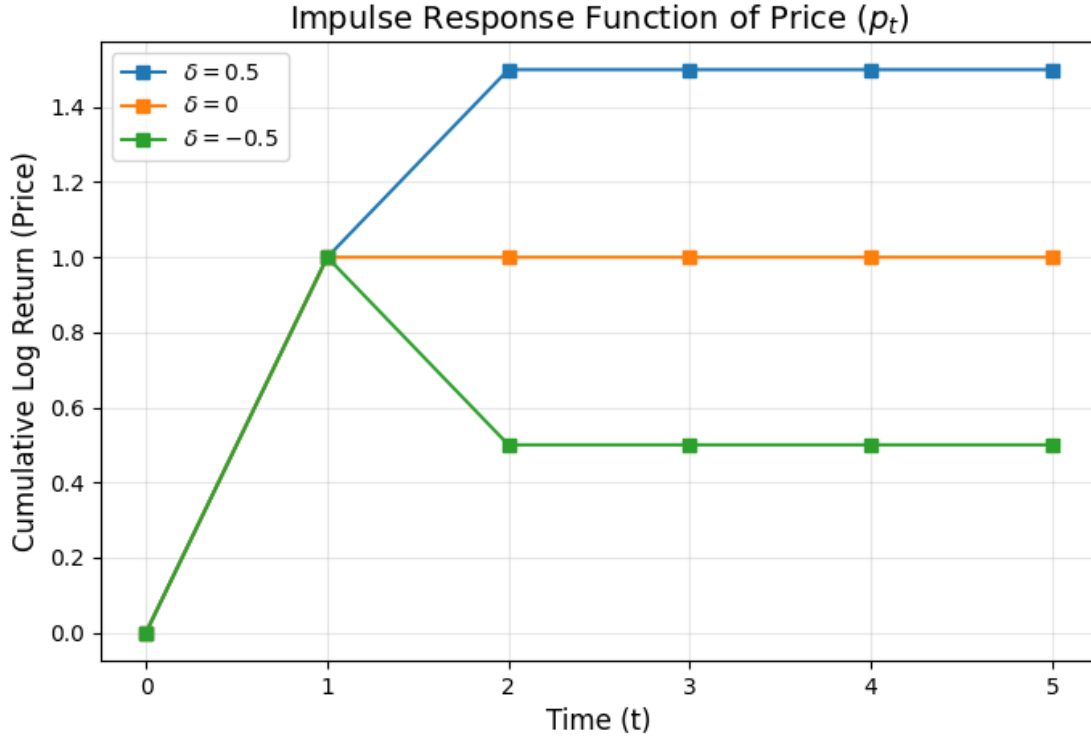


Figure 6: Impulse response function of cumulative log returns to a single $\varepsilon = 1$ shock. As for the returns function, the behaviour from $t = 2$ depends on delta.

From our earlier work, we know that:

- $VAR(r_t) = \sigma_\varepsilon^2(1 + \delta^2)$
- $COV(r_t, r_{t\pm 1}) = \delta\sigma_\varepsilon^2$
- $\forall k \geq 2, \quad COV(r_t, r_{t\pm k}) = 0$

Because returns are correlated, we can't just take the sum of variances. We use this formula:

$$VAR\left(\sum_{i=1}^k r_{t+i}\right) = \sum_{i=1}^k VAR(r_{t+i}) + 2 \sum_{i < j} COV(r_{t+i}, r_{t+j})$$

We have k values, we can create k-1 adjacent pairs. Here we have k-1 adjacent pairs of returns whose covariance isn't 0.

$$\begin{aligned} VAR\left(\sum_{i=1}^k r_{t+i}\right) &= k\sigma_\varepsilon^2(1 + \delta^2) + 2(k-1)\delta\sigma_\varepsilon^2 \\ &= \sigma_\varepsilon^2(k(1 + \delta^2) + 2(k-1)\delta) \end{aligned}$$

We can understand how delta affects long horizon investing. Investing at a $\delta < 0$ allows us to enjoy mean-reversion and obtain more stable returns when investing at long horizons, as total variance becomes smaller when k increases. Mean-reversion favors long-term investors.

On the other hand, we see that increasing δ increases risk and that its effect increases when k is also larger. This means that momentum investing is increasingly risky as a long-term investor, and one-year returns are more attractive.

Prices were downloaded via a yahoo finance API, then converted to monthly series and normalized.

2. Stock Market

2.1. Efficient frontier

2.1.1 Part 1- Two risky assets and one risk-free asset

We now want to evaluate the historical performance and risk-return profiles of the the daily price data for the Vanguard European Stock Index Fund (VEUSX) and the Vanguard Long-Term Bond Fund (VBLIX). Let's analyze the graph.

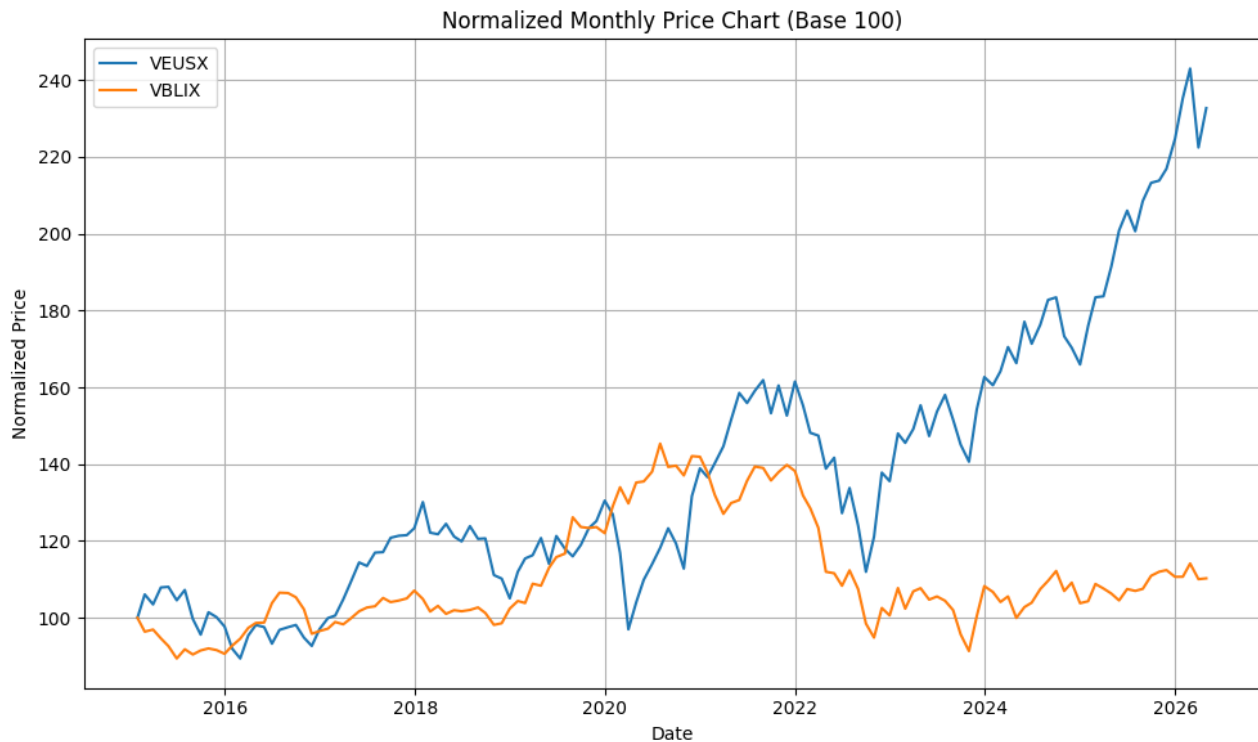


Figure 7: VEUSX and VBLIX indexes from 2015 to the last available data, 29th of april 2029 at the time of the redaction. Both are indexed to 100 at the beginning of 2015.

The graph shows a normalized price comparison between a european stock index and a global bond market index starting from a base of 100 in 2015. While both funds moved relatively closely until 2020, they have since then moved away significantly. VEUSX shows a lot of volatility with a high-growth trajectory, while VBLIX has struggled to maintain its 2020 peaks, reflecting the broader bond bear market of recent years.

2.1.2 Impact of global events

The covid pandemic

We observe a sharp drop in stock prices at the beginning of followed by a strong recovery at the beginning around 2022. due to stimulus measures and investments in digital companies. Simultaneously, bonds saw a significant price spike during the beginning of the crisis. This is called a flight to quality, where investors sell stocks and buy long-term bonds, which drives bond prices up and yields were driven down to near zero.

The War in Ukraine

The Russian invasion of the Ukranian territory marked an increase in inflation in the entire world, especially Europe, where energy prices increased significantly, and an energy shortage

was imminent. This period was also marked by geopolitical instability and the start of aggressive interest rate hikes by central banks to combat rising energy costs and inflation. This caused an important drawdown in stock prices, as increased interest rates usually reduce the quantity of investment.

On the other hand, bonds also dropped significantly, in a very correlated fashion with stocks. This is due to the increase in interest rates by central banks, as increased yields reduce the value of bonds.

Post-2024 Divergence

VEUSX entered a massive bull run starting in late 2023 and continuing through 2026, reaching a normalized price of 240.

At the same time, VBLIX has remained largely stagnant, fluctuating between 100 and 110. Thus, while the equity market has found new growth drivers, mostly with the AI bubble, the long term bond market remains down due to high interest rates.

2.1.3 Matrices of sample returns and variance-covariance

Based on our plot analysis alone, we expect VEUSX to show a higher return and variance than VBLIX.

Sample means of monthly returns

Ticker	Mean (%)
VBLIX	0.12
VEUSX	0.74

Table 1: Sample variance-covariance matrix

Ticker	VBLIX (%)	VEUSX (%)
VBLIX	0.1	0.07
VEUSX	0.07	0.23

Our intuition was correct! Stocks reward investors with a higher return, at a cost of increased risk. Now that we have these values, let's put them to good use and find ourselves a beautiful portfolio allocation.

2.1.4 Portfolio optimization and the Efficient frontier

Having established the individual performance metrics and the covariance between the European stock index and the long-term bond fund, we now shift to the construction of optimal portfolios. To do this, we apply the efficient portfolio theory of (Markowitz, 1952) following the instructions of (Iania & Maes, 2015, p3) .

The Efficient Frontier represents the set of optimal portfolios $w = \{w_1, \dots, w_N\}'$ that offer the highest expected return for a defined level of risk, subject to constraints. For our two-asset portfolio, the expected return R_p is a (2×1) vector, and Σ is the 2×2 covariance matrix of the asset returns, where volatility :

$$\sigma_p = \sqrt{w' \Sigma w} \quad (1)$$

The constraints are:

$$w' \mu = R_p \quad (2)$$

$$w' \iota = 1 \quad (3)$$

Where ι is a vector of ones.

Objective function:

$$\min_w w' \Sigma w$$

From equation 2, we get:

$$R_p = w_1\mu_1 + (1 - w_1)\mu_2$$

From equation 1, we compute $w'\Sigma w$.

$$\begin{aligned} w'\Sigma w &= \begin{bmatrix} w_1 & 1 - w_1 \end{bmatrix} \begin{bmatrix} \sigma_1^2 & Cov_{1,2} \\ Cov_{1,2} & \sigma_2^2 \end{bmatrix} \begin{bmatrix} w_1 \\ 1 - w_1 \end{bmatrix} \\ &= \begin{bmatrix} w_1 & 1 - w_1 \end{bmatrix} \begin{bmatrix} w_1\sigma_1^2 + (1 - w_1)Cov_{1,2} \\ w_1Cov_{1,2} + (1 - w_1)\sigma_2^2 \end{bmatrix} \\ w'\Sigma w &= w_1 \left[w_1\sigma_1^2 + (1 - w_1)Cov_{1,2} \right] + (1 - w_1) \left[w_1Cov_{1,2} + (1 - w_1)\sigma_2^2 \right] \end{aligned}$$

We expand the factors and simplify:

$$w'\Sigma w = w_1^2\sigma_1^2 + (1 - w_1)^2\sigma_2^2 + 2w_1(1 - w_1)Cov_{1,2}$$

And finally, we replace $w'\Sigma w$ in the original equation and we get:

$$\sigma_p = \sqrt{w_1^2\sigma_1^2 + (1 - w_1)^2\sigma_2^2 + 2w_1(1 - w_1)Cov_{1,2}}$$

R_p and σ_p are calculated by varying the weight w_1 from 0 to 1. The resulting curve in the mean-standard deviation plane illustrates the diversification benefit: because the assets are not perfectly correlated, the portfolio risk is lower than the weighted average of the individual risks.

2.1.5 Global minimum variance portfolio (GMVP)

The GMVP is the point on the efficient frontier that provides the lowest possible variance, regardless of the expected return (Iania & Maes, 2015, p9). This is the optimal starting point for the most risk-averse investors. Mathematically, we find the weight w^* by minimizing the variance function

$$w^* = \frac{\Sigma^{-1}\iota'}{\iota'\Sigma^{-1}\iota} \quad (4)$$

which is equal to:

$$w^* = \frac{\sigma_2^2 - Cov_{1,2}}{\sigma_1^2 + \sigma_2^2 - 2Cov_{1,2}}$$

Based on our data, the GMVP results are as follows:

Metric	Value (%)
Weight VEUSX	18.6
Weight VBLIX	81.4
Expected Return	0.24
Standard Deviation	3.1

Table 2: Parameters of the GMV portfolio.

In this case, the GMVP is heavily weighted toward the VBLIX, as its variance is much lower than the other option.

2.1.6 The tangency portfolio and the capital allocation line (CAL)

By introducing a risk-free asset with a monthly rate of $r_f = 0.025\%$, we can identify the tangency portfolio, which maximizes the Sharpe ratio, where

$$Sharpe(w) = \frac{R_p - r_f}{\sigma_p} \quad (5)$$

We compute $Sharpe(w)$ for each portfolio on the efficient frontier and select the vector of weights with the highest results.

Metric	Value (%)
Weight VEUSX	159.41
Weight VBLIX	-59.41
Expected Return	1.13
Standard Deviation	7.1

Table 3: Parameters for the Tangency Portfolio.

The capital allocation line is the line connecting the r_f and the tangency portfolio. This line represents the new efficient frontier when an investor can combine risky assets with risk-free lending or borrowing. Depending on whether the investor wants to short sell government bonds, the CAL stops at the tangency portfolio, or continues infinitely. If he doesn't desire this option, a new constraint is added: $w_i \geq 0 \quad \forall i = 1, \dots, N$.

2.1.7 Plot of the efficient frontier

2.1.8 Part 2- Five risky assets and one risk-free asset

To evaluate the impact of broader diversification, the analysis is expanded from a two-asset framework to a multi-asset universe consisting of five tech equities: Google (GOOG), Cisco (CSCO), Logitech (LOGI), Amazon (AMZN), and Apple (AAPL). The underlying mechanics of our previous optimization remain identical, but the calculations scale into matrix notation to accommodate the larger asset pool ($N = 5$).

Let $\mathbf{w} = [w_1, w_2, w_3, w_4, w_5]^T$ represent the vector of portfolio weights, $\mu = [\mu_1, \mu_2, \mu_3, \mu_4, \mu_5]^T$ be the vector of expected monthly returns, and Σ signify the 5×5 sample variance-covariance matrix. The portfolio expected return (R_p) and portfolio variance (σ_p^2) are mathematically defined as:

$$R_p = \mathbf{w}^T \mu = \sum_{i=1}^5 w_i \mu_i$$

$$\sigma_p^2 = \mathbf{w}^T \Sigma \mathbf{w} = \sum_{i=1}^5 \sum_{j=1}^5 w_i w_j Cov_{i,j}$$

Subject to the budget constraint that all allocations must sum to one: $\mathbf{w}^T \mathbf{1} = \sum_{i=1}^5 w_i = 1$. Regarding GMVP, the optimal weight vector is the same as in equation 4. Similarly, the tangency portfolio weights are derived by maximizing the Sharpe ratio, which can be found in the previous equation 5.

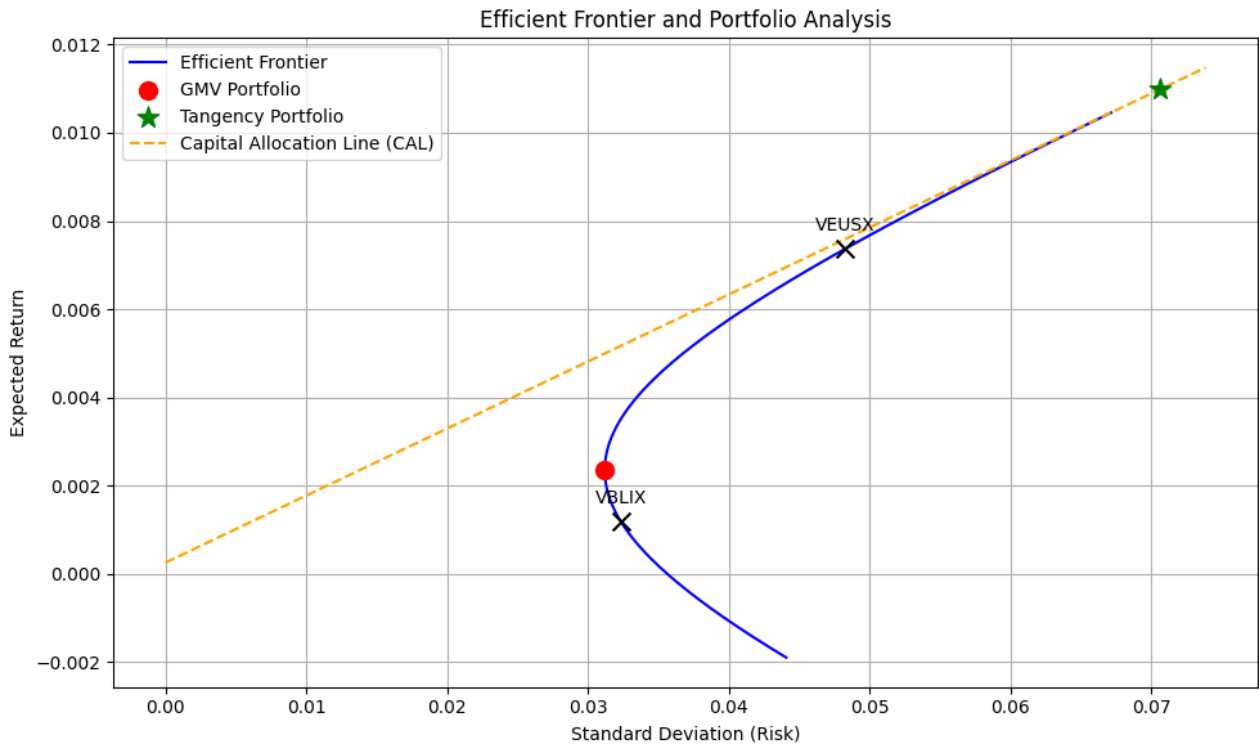


Figure 8: Efficient frontier and capital allocation line. The blue curve represents the efficient frontier generated from VEUSX and VBLIX monthly returns (2015–present). The GMV portfolio and the tangency portfolio are highlighted. The CAL is tangent to the frontier at the tangency portfolio and maximizes the portfolio’s Sharpe ratio.

Table 4: GMV portfolio with 5 assets

Metric	Value (%)
Weight GOOG	21
Weight CSCO	-0.8
Weight LOGI	37.93
Weight AMZN	26.31
Weight AAPL	15.83
Expected Return	1.94
Standard Deviation	5.69

Table 5: Tangency portfolio with 5 assets

Metric	Value (%)
Weight GOOG	25.02
Weight CSCO	9.53
Weight LOGI	16.60
Weight AMZN	34.70
Weight AAPL	14.15
Expected Return	2.1
Standard Deviation	5.91

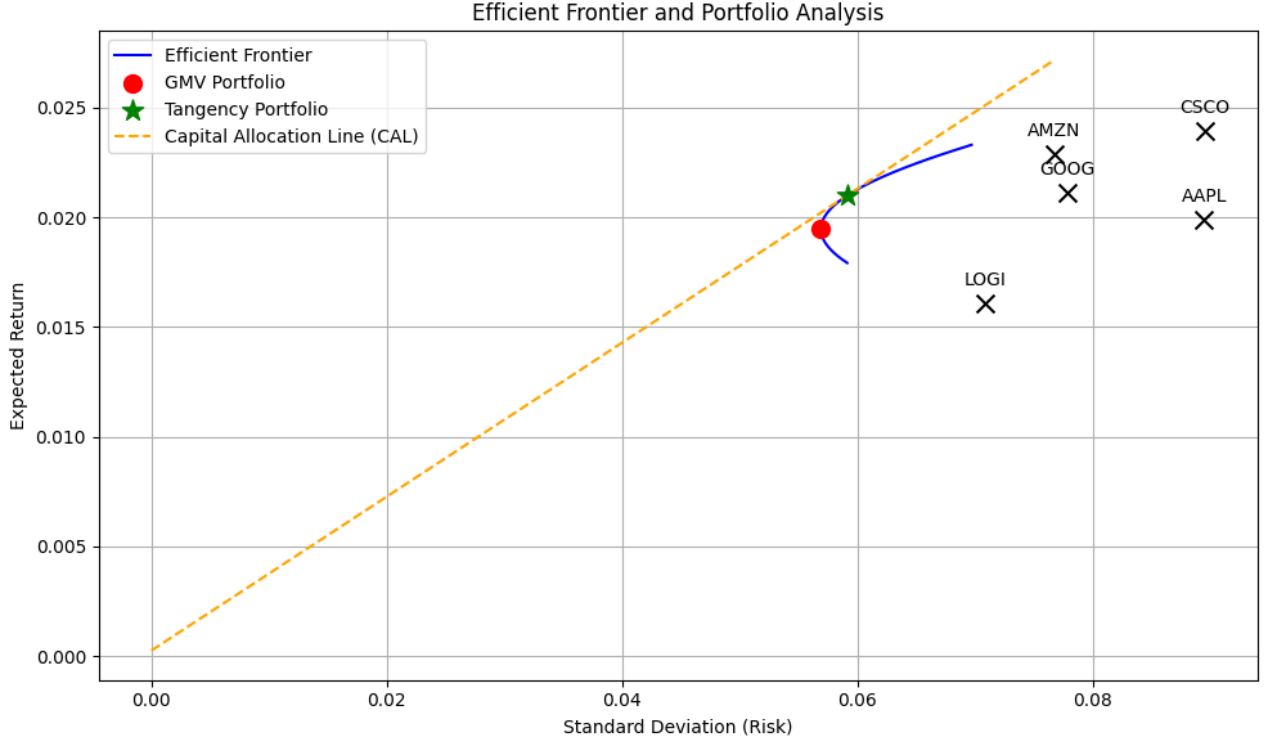


Figure 9: Efficient frontier optimized from the monthly returns of five technology equities: GOOG, CSCO, LOGI, AMZN, and AAPL (2015–present).

2.2. Stock returns and strategies

Now we start by extracting the data from the two CSV files: the 30 Industry Portfolios and the Fama-French 3 Factors model. Observations are taken from July 1927 to March 2026 and divided by 100 to obtain the real values. To focus on the potential deviations from the Capital Asset Pricing Model (CAPM), we calculate the monthly excess returns for each of the 30 industries by subtracting the Risk-Free rate (RF) from each industry's total return:

$$R_{i,t}^e = R_{i,t} - R_{f,t}$$

To implement the industry momentum strategy, we rank the industries dynamically each month based on their trailing performance. For each industry i in month t , we compute the arithmetic average return over the previous 12 months, strictly excluding the current month t to avoid look-ahead bias:

$$\text{Past Average Return}_{i,t} = \frac{1}{12} \sum_{k=1}^{12} R_{i,t-k}$$

Once these averages are calculated, we sort and cross-sectionally rank the 30 industries for that specific month. An industry is assigned a rank from 1 to 30, where a rank of 1 represents the lowest past average return, the loser industry and a rank of 30 represents the highest past average return, the winner. Then, we compute each industry's average rank across the entire sample period.

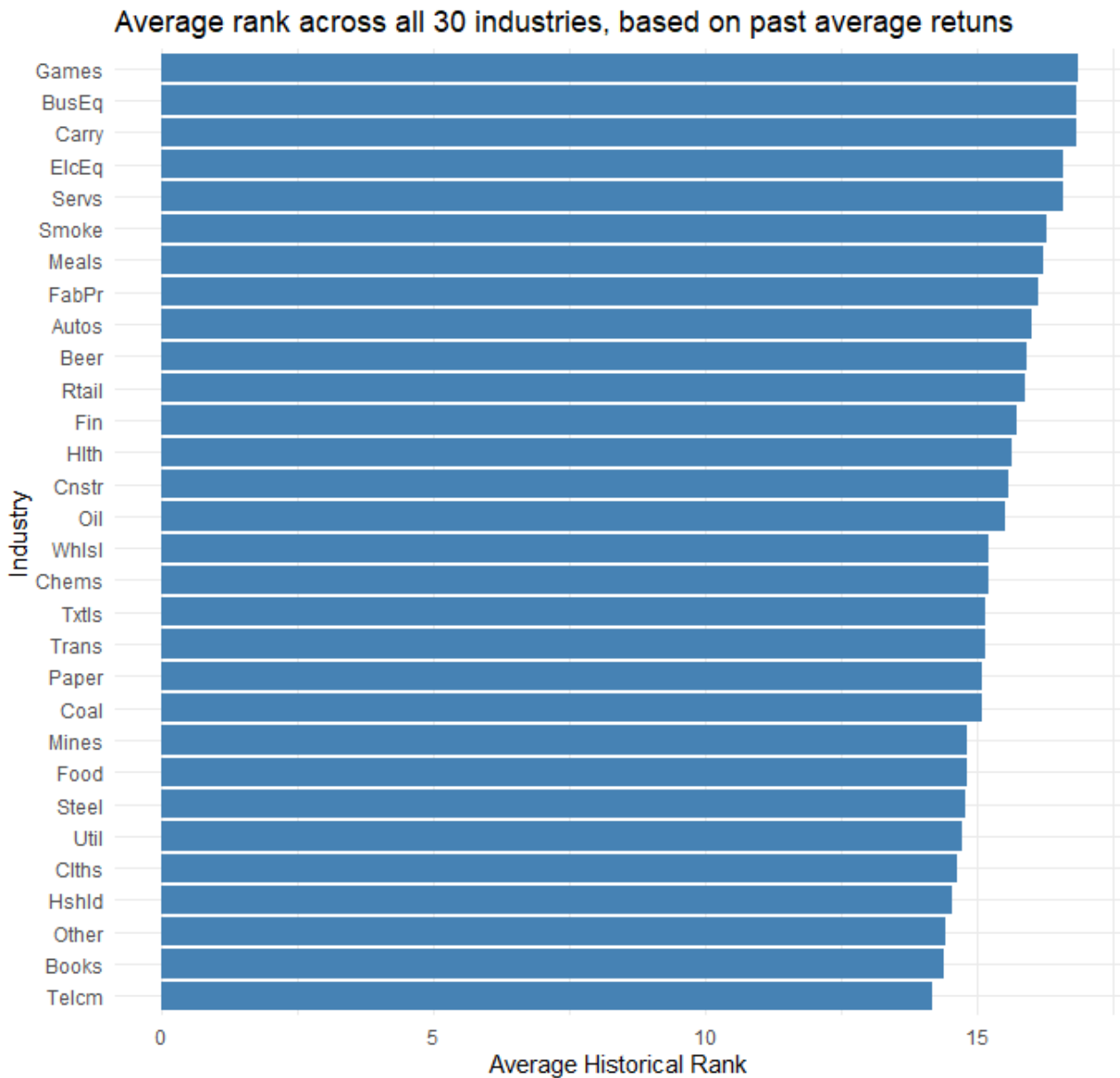


Figure 10: Plot of the average rank for each of the 30 industries from July 1927 to March 2026. Rankings are bounded between 1 (lowest trailing 12-month return) and 30 (highest trailing 12-month return).

The industry with the absolute lowest average rank is Telcm and the industry with the highest average rank is Games.

Industry	Avg rank
Games	14.19778
Telcm	16.85251

Table 6: Average ranks of the winner and loser

As we can see, there isn't really a clear dominant industry here. The average rank from the loser is relatively close to the winner's. This indicates that no single sector structurally dominates or

permanently stays at the bottom of the momentum rankings over a century of financial history.

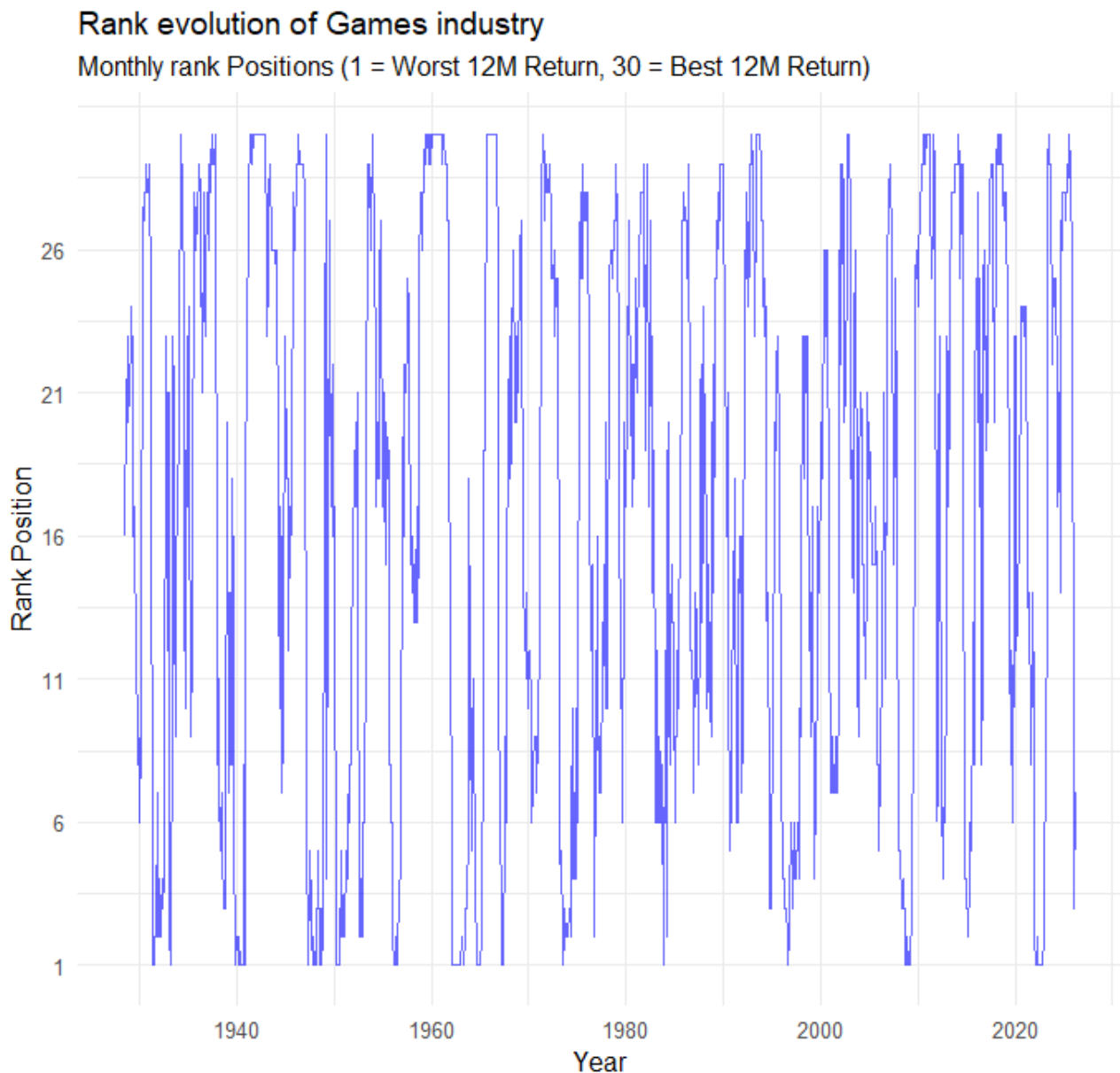


Figure 11: Historical momentum Rank evolution for the Games industry. This plot tracks its shifting monthly rank from 1927 to 2026. The volatile fluctuations across the full vertical axis illustrate the cyclical regime shifts inherent to industry momentum.

The top industries are highly unstable and move around significantly over time. Rather than being a long-term thematic bet on a select few structural winners, industry momentum is a highly dynamic strategy.

Industries regularly rotate from the top quintile (ranks 25–30) down to the bottom quintile (ranks 1–5) and vice versa. In consequence, an investor should expect exceptionally high turnover from this strategy. Because the composition of the top winner and bottom loser portfolio rotates continuously, executing this strategy in practice requires extensive monthly rebalancing, which will incur substantial transaction costs.

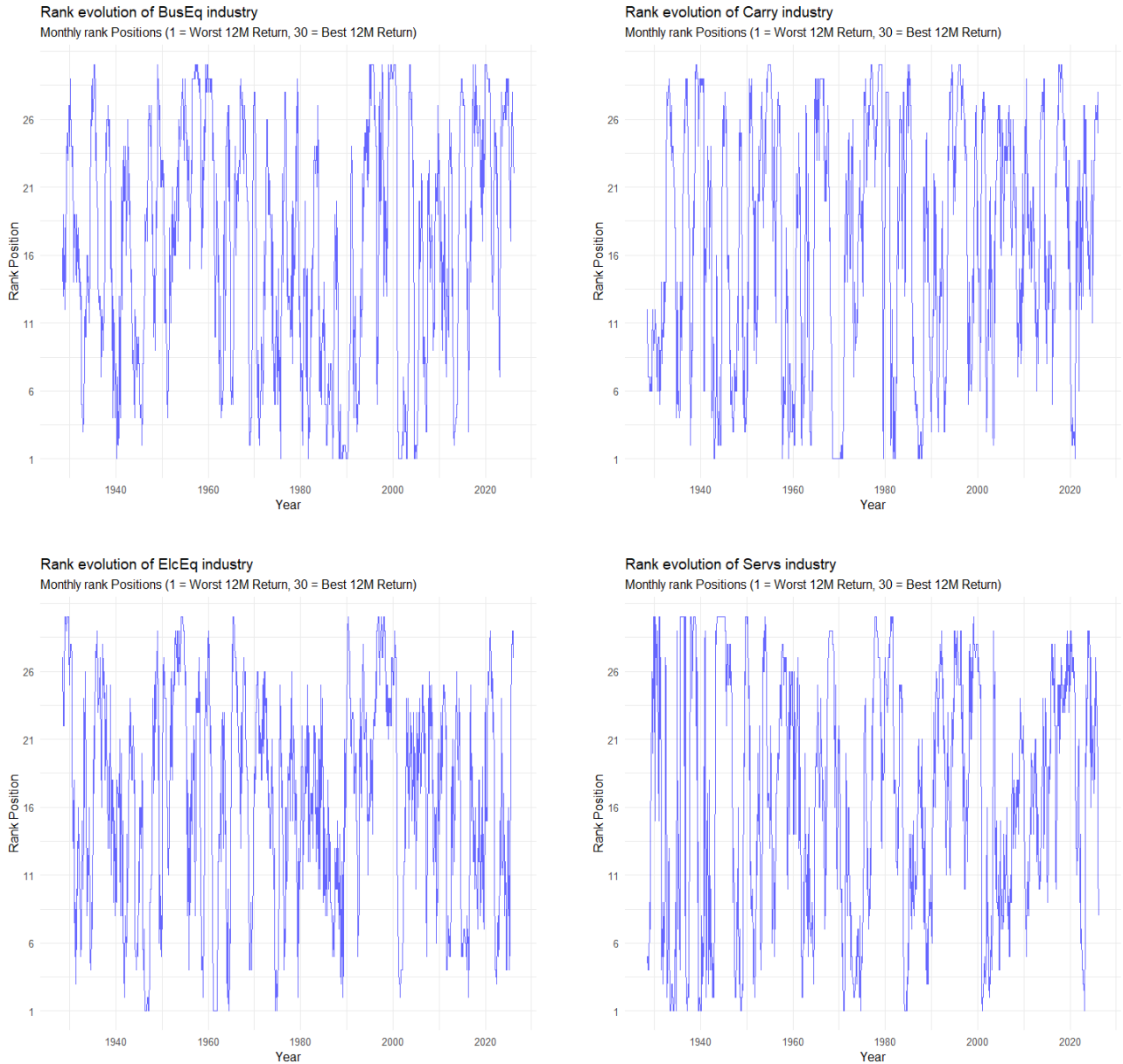


Figure 12: Plots of the top 4 industries after Games. Clearly, industry momentum strategy would be very costly to implement, due to exorbitant transaction costs.

2.2.1 Winner and loser portfolios

To evaluate the performance of the industry momentum strategy, we construct two equally weighted portfolios each month: the Winner Portfolio, which consists of the 15 industries with the highest past 12-month returns, and the Loser Portfolio, which contains the 15 industries with the lowest past returns. For both portfolios, we calculate the monthly average excess return and volatility to derive the monthly Sharpe ratios, which we scale to an annualized basis ($SR_{\text{annualized}} = SR_{\text{monthly}} \times \sqrt{12}$). Finally, we compute the annualized Sharpe ratio of the overall market index over the same time horizon (July 1927 to March 2026) to serve as a benchmark, allowing us to determine whether tilting into industry momentum generates superior risk-adjusted performance relative to a passive market buy-and-hold strategy. The annualized Sharpe ratio of the benchmark is equal to 0.4314, which sits between the SR from the winner and the loser.

Performance Metric	Winner Portfolio	Loser Portfolio
Avg Monthly Excess Return	0.0094	0.0052
Monthly Standard Deviation	0.0557	0.0597
Monthly Sharpe Ratio	0.1687	0.0871
Annualized Sharpe Ratio	0.5844	0.3019

Table 7: Winner vs loser portfolios performance metrics Comparison

2.2.2 Cumulative returns comparison

We compute the cumulative return of the winner portfolio, the loser portfolio, the long/short ind-mom portfolio, and the market. For this, we add back the risk-free rate to our excess returns. Then we plot the results to visualize the results obtained for each strategy after a century of investing.

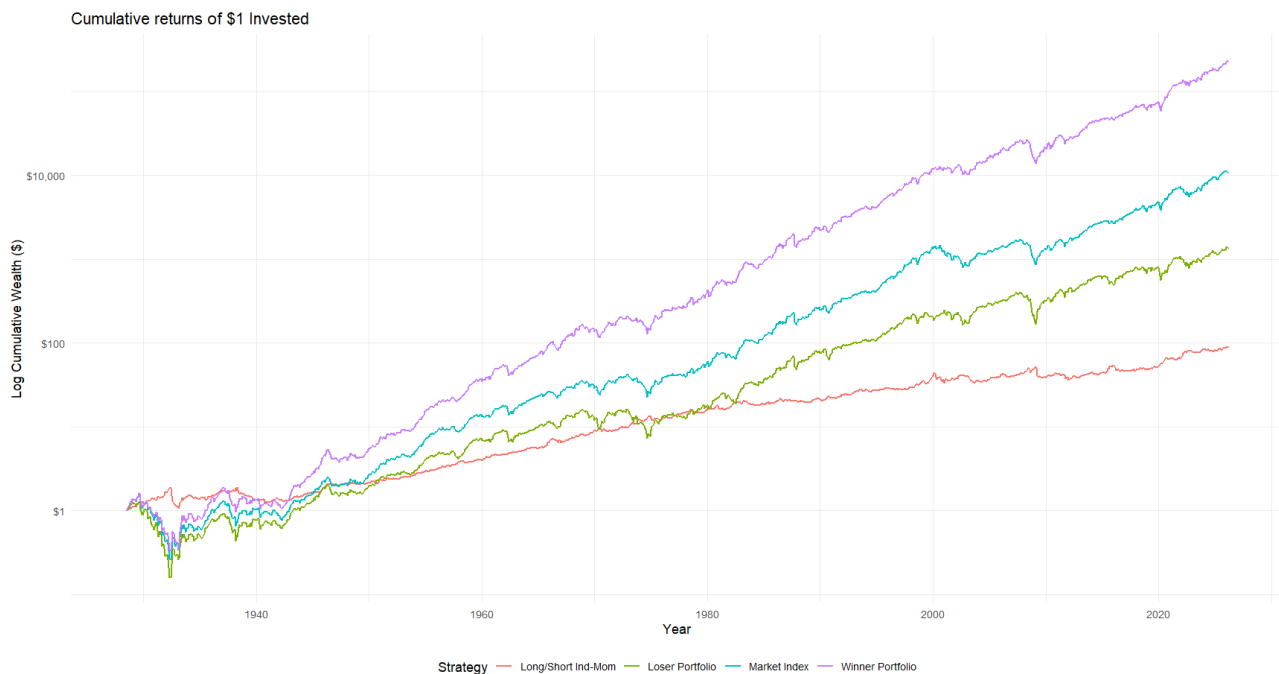


Figure 13: Cumulative returns of \$1 invested, comparing different industry momentum strategies and a market benchmark

2.2.3 The 2009 crash

The three consecutive months where the industry momentum strategy suffered its most severe, compounding losses are March, April, and May 2009. When looking further into the results, we observe that the overall market had an incredible bull run during the same quarter, getting 9%, 10% and 5% returns from march to may of that year. What's even more striking are the returns from the loser portfolio, 11%, 26% and 7%. These are absolutely incredible results, and they mark the end of the 2008 stock market crash and the beginning of an incredible bull run that lasted until the covid pandemic.

The huge drawdown way the industry momentum portfolio has faced is due to the way it's constructed. It is a strategy that buys the winner industries and sells the losers. In this particular case, their short position became massively unprofitable, leading to huge losses.

Date	Ind_Mom_Total	Market_Total	Winner_Total	Loser_Total
200901	0.0406	-0.0809	-0.0696	-0.1100
200902	0.0463	-0.1010	-0.0877	-0.1340
200903	-0.0394	0.0903	0.0748	0.1140
200904	-0.1840	0.1020	0.0759	0.2600
200905	-0.0445	0.0521	0.0332	0.0777
200906	0.0187	0.0043	-0.000960	-0.0197
200907	-0.00763	0.0775	0.1030	0.1100
200908	-0.0257	0.0334	0.0198	0.0455
200909	-0.0122	0.0409	0.0467	0.0590
200910	0.000120	-0.0253	-0.0302	-0.0303
200911	0.0115	0.0563	0.0692	0.0578
200912	0.0335	0.0281	0.0580	0.0245

D

Table 8: Industry momentum performance in 2009, compared with the stock market

2.2.4 Similar cases

To see when the industry momentum faced similar losses to its 2009 crash, we select the 10 worst months for the strategy in its existence, and compare them to the loser portfolio.

As we can see, every time the strategy suffers massive losses, we can blame that on a significant positive return for the loser industries. It seems like there is a trend where loser industries are significantly underperforming during a crisis, as investors flee to safer investments, and suddenly jump back up to mark the end of the crisis. We can see this after the Dot-com bubble, the 1930s recession, the 1970s recession and the covid pandemic.

Date	Ind_Mom_Total	Market_Total	Winner_Total	Loser_Total
193207	-0.1870	0.3360	0.2520	0.4390
200904	-0.1840	0.1020	0.0759	0.2600
193208	-0.1300	0.3720	0.3620	0.4920
193806	-0.1300	0.2390	0.1750	0.3050
200101	-0.1030	0.0379	-0.0271	0.0758
200004	-0.0831	-0.0593	-0.0424	0.0407
201110	-0.0761	0.1130	0.0955	0.1720
197501	-0.0741	0.1420	0.1430	0.2170
193209	-0.0716	-0.0291	-0.0587	0.0129
202301	-0.0683	0.0696	0.0501	0.1180

Table 9: Top 10 Worst Months for Ind-Mom

3. Bond Market

In this section, we test the Expectation Hypothesis (EH) of the interest rates using the test developed by [Campbell and Shiller, 1991](#) using the data provided by [Liu and Wu, 2021](#) and following the instructions in the slides of [Iania, n.d.](#)

3.1. Test 1: Test of long yield changes over the life of the short bond

3.1.1 Model specification

The first test evaluates whether the yield spread between a longer-term and a shorter-term bond can correctly predict the short-term change in the yield of the longer-term bond. According to the Expectations Hypothesis (EH), assuming a time-invariant risk premium, the relationship between long and short yields can be formalized as:

$$y_{n-m,t+m} - y_{n,t} = \alpha_0 + \alpha_1 \frac{m}{n-m} (y_{n,t} - y_{m,t}) + \epsilon_{t+m}$$

In this regression equation, $y_{n,t}$ is the yield of a bond with maturity n at time t , and m is the shorter maturity horizon. The term $\frac{m}{n-m}(y_{n,t} - y_{m,t})$ represents the theoretical spread. If the EH holds true, the slope coefficient α_1 must equal exactly 1, and the intercept α_0 should capture the constant term premium.

3.1.2 Emprirical results

n	m	$\hat{\alpha}_1$	SE($\hat{\alpha}_1$)	$Frac_{dir}$	$Frac_t$
2	1	-1***	0.0005	0.833	0.000
3	1	0***	0.0000	0.525	0.000
3	2	-5133.26***	629.0108	0.411	0.001
4	1	0***	0.0000	0.529	0.000
4	2	-0.326***	0.4454	0.492	0.042
4	3	-3658.16***	541.9269	0.396	0.001
6	1	0***	0.0000	0.531	0.000
6	2	-0.518***	0.4400	0.498	0.022
6	3	-0.36***	0.3676	0.488	0.008
6	4	-0.236***	0.3132	0.486	0.004
9	1	0***	0.0001	0.531	0.000
9	2	-0.709***	0.4418	0.499	0.015
9	3	-0.561***	0.3786	0.488	0.005
9	4	-0.421***	0.3480	0.485	0.006
9	6	-0.221***	0.3598	0.500	0.007
12	1	0***	0.0001	0.528	0.000
12	2	-0.876***	0.4653	0.503	0.008
12	3	-0.735***	0.4198	0.490	0.004
12	4	-0.594***	0.3992	0.478	0.004
12	6	-0.384***	0.4061	0.503	0.006
24	1	0***	0.0001	0.501	0.000
24	2	-1.19***	0.5991	0.497	0.004
24	3	-1.048***	0.5503	0.488	0.001
24	4	-0.968***	0.5134	0.511	0.001
24	6	-0.821***	0.4772	0.514	0.000
24	12	-0.502***	0.4276	0.487	0.006
36	1	0***	0.0002	0.509	0.000
36	2	-1.52***	0.7290	0.498	0.004
36	3	-1.359***	0.6616	0.494	0.001
36	4	-1.255***	0.5958	0.509	0.000
36	6	-1.083***	0.5131	0.512	0.000
36	12	-0.78***	0.4811	0.498	0.002
48	1	0***	0.0003	0.507	0.000
48	2	-1.838***	0.8058	0.499	0.001
48	3	-1.642***	0.7206	0.506	0.001
48	4	-1.544***	0.6309	0.515	0.000
48	6	-1.397***	0.5328	0.504	0.000
48	12	-1.096***	0.5218	0.499	0.002

Table 10: Test 1 Results: Long yield changes over life of short bond

3.1.3 Results analysis

The empirical results contradict the core predictions of the EH. Across almost all combinations of maturities m and n , the estimated slope coefficients ($\hat{\alpha}_1$) are systematically below 1, frequently turning negative, and are statistically significantly different from the theoretical value of 1. Under the logic of the EH, an upward-sloping yield curve implies that the long rate must rise over the short term. This anticipated rise inflicts a capital loss on the long bond, thereby equalizing its holding period return with that of the short-term bond. However, the negative $\hat{\alpha}_1$ coefficients obtained in the data indicate the exact opposite behavior: when the yield spread is unusually high, the yield on the longer-term bond actually tends to fall over the short term. Furthermore, there is a clear monotonic pattern across the maturity spectrum. As the maturity of the long bond (n) increases, the magnitude of the rejection typically becomes much stronger, yielding increasingly negative coefficients. To ensure these rejections are not simply artifacts of asymptotic standard errors in overlapping samples, the bootstrap simulation metrics ($Frac_{dir}$ and $Frac_t$) provide a finite-sample robustness check. The highly elevated $Frac_t$ values confirm that the empirical t-statistics are rarely exceeded in simulated samples that enforce the EH, heavily reinforcing the rejection of the null hypothesis.

3.2. Test 2: Test of short yield changes over the life of the long bond

3.2.1 Model Specification

The second test of the Expectations Hypothesis (EH) flips the perspective of Test 1. Instead of predicting the short-term change in a long-term yield, this test evaluates whether the current yield spread correctly forecasts the long-term changes in short-term interest rates.

According to the EH, the long-term yield is an average of current and expected future short-term yields, plus a constant term premium (Φ_n). By subtracting the current short-term yield from both sides of this relationship, we obtain the "perfect-foresight spread"—the theoretical spread that would exist if investors had perfect foresight of future short rates.

As outlined in the assignment instructions, for a 1-period short rate, the testable regression equation is:

$$\frac{1}{n} \sum_{i=0}^{n-1} y_{1,t+i} - y_{1t} = \beta_0 + \beta_1(y_{nt} - y_{1t}) + \epsilon_{t+n-1}$$

More generally, for any short maturity m where n is an integer multiple of m (such that $k = n/m$), the regression tests whether the actual yield spread $S_t^{(n,m)}$ predicts the perfect-foresight spread $S_t^{(n,m)*}$:

$$\left(\frac{1}{k} \sum_{i=0}^{k-1} y_{m,t+i \cdot m} \right) - y_{m,t} = \beta_0 + \beta_1(y_{n,t} - y_{m,t}) + \epsilon_{t+n-m}$$

Under the null hypothesis of the Expectations Hypothesis, the slope coefficient β_1 must equal exactly 1, and the intercept β_0 represents the negative of the constant term premium.

3.2.2 Empirical results

n	m	$\hat{\beta}_1$	$SE(\hat{\beta}_1)$	Frac_dir	Frac_t
2	1	0***	0.0002	0.833	0.000
3	1	0***	0.0004	0.872	0.000
4	1	0***	0.0005	0.897	0.000
4	2	0.337***	0.2365	0.492	0.043
6	1	0***	0.0006	0.946	0.000
6	2	0.381***	0.1887	0.482	0.012
6	3	0.32***	0.1928	0.488	0.008
9	1	0***	0.0004	0.972	0.000
9	3	0.373***	0.1739	0.490	0.008
12	1	0***	0.0001	0.386	0.000
12	2	0.45***	0.1829	0.482	0.028
12	3	0.412***	0.1793	0.505	0.015
12	4	0.341***	0.1760	0.492	0.002
12	6	0.308***	0.2053	0.503	0.007
24	1	0***	0.0001	0.377	0.000
24	2	0.562**	0.2096	0.492	0.060
24	3	0.529**	0.2111	0.495	0.067
24	4	0.491**	0.2125	0.495	0.047
24	6	0.423***	0.2178	0.510	0.022
24	12	0.249***	0.2170	0.487	0.006
36	1	0***	0.0001	0.370	0.000
36	2	0.654	0.2428	0.489	0.230
36	3	0.629	0.2431	0.480	0.198
36	4	0.603	0.2444	0.471	0.157
36	6	0.554*	0.2514	0.494	0.130
36	12	0.431**	0.2711	0.511	0.083
48	1	0***	0.0001	0.359	0.000
48	2	0.697	0.2497	0.499	0.293
48	3	0.694	0.2432	0.495	0.266
48	4	0.689	0.2386	0.513	0.274
48	6	0.67	0.2374	0.508	0.239
48	12	0.576*	0.2544	0.483	0.158

Table 11: Test 2 results: Short yield changes over the life of long bond

3.2.3 Results analysis

The empirical results for Test 2 present a striking contrast to the findings in Test 1. While Test 1 yielded consistently negative coefficients, the $\hat{\beta}_1$ slope coefficients in Test 2 are almost universally positive. However, the validity of the Expectations Hypothesis under this test is highly dependent on the forecast horizon.

- For shorter maturities (e.g., n up to 12 months), the $\hat{\beta}_1$ coefficients remain substantially below 1. Despite being positive, they are statistically distinguishable from the theoretical value of 1, leading to a rejection of the EH at conventional significance levels.
- As the maturity of the long bond (n) increases toward the higher end of the spectrum (e.g., 24 to 48 months), the slope coefficients systematically rise toward 1. At these longer horizons, the Expectations Hypothesis is generally not rejected by the data.

Because the dependent variable in this regression is constructed using data up to $n - m$ periods into the future, the error terms are highly autocorrelated. The Newey-West standard errors and the bootstrap $Frac_t$ metrics are critical here to ensure that the failure to reject the EH at the long end is not merely a consequence of inflated standard errors due to overlapping samples.

3.3. Conclusion of both tests

The contrasting results of Test 1 and Test 2 confirm the apparent paradox famously documented by [Campbell and Shiller, 1991](#). The slope of the term structure almost always gives a forecast in the wrong direction for the short-term change in the yield of the longer bond (as seen in Test 1), but simultaneously provides a forecast in the right direction for long-term changes in short rates (as seen in Test 2). This implies that while the yield curve does contain valuable information about the future path of short-term interest rates, long-term yields overreact or are overly sensitive to transient shocks in short-term rates, likely driven by time-varying term premia.

References

- Campbell, J. Y., & Shiller, R. J. (1991). Yield spreads and interest rate movements: A bird's eye view. *The Review of Economic Studies*, 58(3), 495–514. <https://doi.org/10.2307/2298008>
- Google. (2026). Gemini [Large language model: Helped with paragraph readability, understanding of mathematical formulas, and LaTeX syntax].
- Iania, L. (n.d.). The empirics of financial markets: The term structure of interest rates.
- Iania, L., & Maes, S. (2015). Portfolio theory: Empirics of financial markets.
- Liu, Y., & Wu, J. C. (2021). Reconstructing the yield curve. *Journal of Financial Economics*, 142(3), 1395–1425.
- Markowitz, H. (1952). Portfolio selection. *The Journal of Finance*.

A. Source Code

A.1. Python Script for question 1: Standard deviation bands, prediction cones and simulated paths

Listing 1: Plots for question 1

```

1 #####Author : Jean de Biolley #####3
2 ##### Q1.c.i #####
3
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 rho = 0.95
8 x1 = 1
9 t_max = 20
10
11 t_values = np.arange(1, t_max + 1)
12
13 conditional_means = [x1 * (rho**(t - 1)) for t in t_values]
14
15 plt.figure(figsize=(10, 6))
16 plt.plot(t_values, conditional_means, marker='o', linestyle='--', color='b',
17         label=f'Conditional Mean (rho={rho})')
18
19 plt.title('Conditional Mean  $E_1(x_t)$  for AR(1) Process')
20 plt.xlabel('Time (t)')
21 plt.ylabel('Expected Value')
22 plt.xticks(t_values)
23 plt.grid(True, linestyle='--', alpha=0.7)
24 plt.axhline(0, color='black', linewidth=0.8) # Long-run mean
25 plt.legend()
26
27 plt.show()
28
29 #####3 Q1.c.ii #####
30
31 sigma_eps = 0.25
32
33 cond_mean = x1 * (rho**(t_values - 1))
34
35 cond_var = (sigma_eps**2) * (1 - rho**(2 * (t_values - 1))) / (1 - rho**2)
36 cond_std = np.sqrt(cond_var)
37
38 plt.figure(figsize=(10, 6))
39 plt.plot(t_values, cond_mean, marker='o', label='Conditional Mean  $E_1(x_t)$ ',
40         color='blue')
41
42 plt.fill_between(t_values, cond_mean - cond_std, cond_mean + cond_std,
43                 color='blue', alpha=0.15, label='$\pm 1$ Cond. Std. Dev. Band')
44
45 plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
46 plt.title('AR(1) Conditional Mean and  $\pm 1$  Standard Deviation Bands')
47 plt.xlabel('Time (t)')
48 plt.ylabel('$x_t$')
49 plt.xticks(t_values)
50 plt.grid(True, linestyle=':', alpha=0.6)

```

```

50 plt.legend()
51
52 plt.show()
53
54 ##### Q1.c.iii #####
55
56
57 k95 = 1 / np.sqrt(0.05)
58 k90 = 1 / np.sqrt(0.10)
59
60 plt.figure(figsize=(10, 6))
61
62 plt.plot(t_values, cond_mean, marker='o', color='blue', label='Conditional Mean')
63 plt.fill_between(t_values, cond_mean - cond_std, cond_mean + cond_std,
64                  color='blue', alpha=0.15, label='$\pm 1$ Cond. Std. Dev. Band')
65
66 plt.fill_between(t_values, cond_mean - k95*cond_std, cond_mean + k95*cond_std,
67                  color='red', alpha=0.1, label='Chebyshev 95% Band')
68
69 plt.fill_between(t_values, cond_mean - k90*cond_std, cond_mean + k90*cond_std,
70                  color='red', alpha=0.2, label='Chebyshev 90% Band')
71
72 plt.title('AR(1) Prediction Cones (Chebyshev Inequality)')
73 plt.xlabel('Time (t)')
74 plt.ylabel('$x_t$')
75 plt.axhline(0, color='black', linestyle='--', alpha=0.5)
76 plt.xticks(t_values)
77 plt.grid(True, linestyle=':', alpha=0.6)
78 plt.legend()
79
80 plt.show()
81
82 ##### Q1.c.iv #####
83
84 num_simulations = 20
85
86 z95 = 1.96 # 95% confidence
87 z90 = 1.645 # 90% confidence
88
89 all_sims = np.zeros((num_simulations, t_max))
90 all_sims[:, 0] = x1
91
92 for i in range(num_simulations):
93     for t in range(1, t_max):
94         epsilon = np.random.normal(0, sigma_eps)
95         all_sims[i, t] = rho * all_sims[i, t-1] + epsilon
96
97 plt.figure(figsize=(12, 7))
98
99 plt.fill_between(t_values, cond_mean - z95*cond_std, cond_mean + z95*cond_std,
100                  color='blue', alpha=0.1, label='Normal 95% Prediction Cone')
101 plt.fill_between(t_values, cond_mean - z90*cond_std, cond_mean + z90*cond_std,

```

```

102         color='blue', alpha=0.2, label='Normal 90% Prediction Cone')
103
104     for i in range(num_simulations):
105         plt.plot(t_values, all_sims[i, :], color='black', alpha=0.2, linewidth=1)
106
107     plt.plot(t_values, all_sims[0, :], color='black', alpha=0.6, linewidth=1,
108             label='Simulated Paths ($x_t$)')
109     plt.plot(t_values, cond_mean, marker='o', color='blue', linewidth=2,
110             label='Conditional Mean $E_1(x_t)$')
111
112     plt.title(f'AR(1) Simulations vs. Theoretical Mean and Normal Prediction
113             Cones\n($\rho={rho}$, $\sigma_{\epsilon}={sigma\_eps}$)')
114     plt.xlabel('Time (t)')
115     plt.ylabel('$x_t$')
116     plt.axhline(0, color='black', linestyle='--', alpha=0.5)
117     plt.xticks(t_values)
118     plt.grid(True, linestyle=':', alpha=0.6)
119     plt.legend(loc='upper right')
120
121     plt.show()

```

A.2. Python script: plots for the IRF on a MA(1) process

Listing 2: Plots for the IRF

```

1
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 deltas = [0.5, 0, -0.5]
6 t_max = 5
7 t = np.arange(0, t_max + 1)
8
9
10 epsilon = np.zeros(t_max + 2)
11 epsilon[1] = 1
12
13 def get_ma1_irf(delta):
14     r = np.zeros(len(t))
15     for i in range(len(t)):
16         idx = t[i]
17         val = epsilon[idx] + delta * epsilon[idx-1] if idx > 0 else 0
18         r[i] = val
19     return r
20
21 # Plot 1: Return IRF
22 plt.figure(figsize=(8, 5))
23 for d in deltas:
24     r_irf = get_ma1_irf(d)
25     plt.plot(t, r_irf, marker='o', label=rf'$\delta = {d}$')
26

```



```

27 plt.title('Impulse Response Function of Returns ($r_t$)', fontsize=14)
28 plt.xlabel('Time (t)', fontsize=12)
29 plt.ylabel('Return Response', fontsize=12)
30 plt.xticks(t)
31 plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
32 plt.legend()
33 plt.grid(True, alpha=0.3)
34 plt.savefig('return_irf.png')
35 plt.close()
36
37 # Plot 2: Price IRF (Cumulative Log Returns)
38 plt.figure(figsize=(8, 5))
39 for d in deltas:
40     r_irf = get_ma1_irf(d)
41     p_irf = np.cumsum(r_irf)
42     plt.plot(t, p_irf, marker='s', label=rf'$\delta = {d}$')
43
44 plt.title('Impulse Response Function of Price ($p_t$)', fontsize=14)
45 plt.xlabel('Time (t)', fontsize=12)
46 plt.ylabel('Cumulative Log Return (Price)', fontsize=12)
47 plt.xticks(t)
48 plt.legend()
49 plt.grid(True, alpha=0.3)
50 plt.savefig('price_irf.png')
51 plt.close()

```

A.3. Python script for the efficient frontier and capital allocation line: 2 assets

Listing 3: Efficient frontier: 2 assets

```

1 import yfinance as yf
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 tickers = ['VEUSX', 'VBLIX']
7 start_date = '2015-01-01'
8
9 # Download with auto_adjust to avoid the KeyError
10 data = yf.download(tickers, start=start_date, auto_adjust=True)['Close']
11
12 # The rest of your logic remains the same
13 monthly_prices = data.resample('BM').last()
14 monthly_returns = monthly_prices.pct_change().dropna()
15
16 print("First 5 rows of monthly returns:\n", monthly_returns.head())
17
18 # Compute the vector of sample means
19 sample_means = monthly_returns.mean()
20 print("\nSample Means of Monthly Returns:\n", sample_means)

```

```

21
22 # Compute the matrix of sample variances and covariances
23 sample_cov_matrix = monthly_returns.cov()
24 print("\nSample Variance-Covariance Matrix of Monthly Returns:\n",
      sample_cov_matrix)
25
26 # Normalize prices (base = 100 at the start date)
27 normalized_prices = (monthly_prices / monthly_prices.iloc[0]) * 100
28
29 # Plot the normalized price chart
30 plt.figure(figsize=(10, 6))
31 for ticker in tickers:
32     plt.plot(normalized_prices.index, normalized_prices[ticker], label=ticker)
33
34 plt.title('Normalized Monthly Price Chart (Base 100)')
35 plt.xlabel('Date')
36 plt.ylabel('Normalized Price')
37 plt.legend()
38 plt.grid(True)
39 plt.tight_layout()
40
41 # Mean returns and variances-covariances of the assets
42 asset1 = 'VEUSX'
43 asset2 = 'VBLIX'
44
45 mean1 = sample_means[asset1]
46 mean2 = sample_means[asset2]
47
48 var1 = sample_cov_matrix.loc[asset1, asset1]
49 var2 = sample_cov_matrix.loc[asset2, asset2]
50 cov12 = sample_cov_matrix.loc[asset1, asset2]
51
52 # Efficient frontier
53 weights1 = np.linspace(-0.5, 1.5, 200)
54 port_returns = weights1 * mean1 + (1 - weights1) * mean2
55 port_vars = (weights1**2) * var1 + ((1 - weights1)**2) * var2 + 2 * weights1 * (1 -
      weights1) * cov12
56 port_stdevs = np.sqrt(port_vars)
57
58 # Global Minimum Variance (GMV) Portfolio
59 w1_gmv = (var2 - cov12) / (var1 + var2 - 2 * cov12)
60 w2_gmv = 1 - w1_gmv
61
62 gmv_mean = w1_gmv * mean1 + w2_gmv * mean2
63 gmv_var = (w1_gmv**2) * var1 + (w2_gmv**2) * var2 + 2 * w1_gmv * w2_gmv * cov12
64 gmv_stdev = np.sqrt(gmv_var)
65
66 print("\n--- Global Minimum Variance Portfolio ---")
67 print(f"Weight {asset1}: {w1_gmv:.4f}")
68 print(f"Weight {asset2}: {w2_gmv:.4f}")
69 print(f"Expected Return: {gmv_mean:.6f}")
70 print(f"Standard Deviation: {gmv_stdev:.6f}")

```

```

71
72 # Tangency Portfolio
73 rf = 0.00025
74 num1 = (mean1 - rf) * var2 - (mean2 - rf) * cov12
75 den = (mean1 - rf) * var2 + (mean2 - rf) * var1 - (mean1 - rf + mean2 - rf) * cov12
76 w1_tan = num1 / den
77 w2_tan = 1 - w1_tan
78
79 tan_mean = w1_tan * mean1 + w2_tan * mean2
80 tan_var = (w1_tan**2) * var1 + (w2_tan**2) * var2 + 2 * w1_tan * w2_tan * cov12
81 tan_stdev = np.sqrt(tan_var)
82
83 print("\n--- Tangency Portfolio ---")
84 print(f"Weight {asset1}: {w1_tan:.4f}")
85 print(f"Weight {asset2}: {w2_tan:.4f}")
86 print(f"Expected Return: {tan_mean:.6f}")
87 print(f"Standard Deviation: {tan_stdev:.6f}")
88
89 # Capital Allocation Line (CAL)
90 cal_stdevs = np.linspace(0, max(port_stdevs)*1.1, 100)
91 sharpe_ratio = (tan_mean - rf) / tan_stdev
92 cal_returns = rf + sharpe_ratio * cal_stdevs
93
94 # Plot Efficient Frontier
95 plt.figure(figsize=(10, 6))
96 plt.plot(port_stdevs, port_returns, label='Efficient Frontier', color='blue')
97 plt.scatter(gmv_stdev, gmv_mean, color='red', marker='o', s=100, label='GMV
    Portfolio', zorder=5)
98 plt.scatter(tan_stdev, tan_mean, color='green', marker='*', s=150, label='Tangency
    Portfolio', zorder=5)
99 plt.plot(cal_stdevs, cal_returns, label='Capital Allocation Line (CAL)',
    color='orange', linestyle='--')
100
101 # Plot the individual assets
102 plt.scatter([np.sqrt(var1), np.sqrt(var2)], [mean1, mean2], color='black',
    marker='x', s=100, zorder=5)
103 plt.annotate(asset1, (np.sqrt(var1), mean1), textcoords="offset points",
    xytext=(0,10), ha='center')
104 plt.annotate(asset2, (np.sqrt(var2), mean2), textcoords="offset points",
    xytext=(0,10), ha='center')
105
106 plt.title('Efficient Frontier and Portfolio Analysis')
107 plt.xlabel('Standard Deviation (Risk)')
108 plt.ylabel('Expected Return')
109 plt.legend()
110 plt.grid(True)
111 plt.tight_layout()
112 plt.show()

```

A.4. Python script for efficient frontier and CAL: 5 assets

Listing 4: Efficient frontier: 5 assets

```

1 import yfinance as yf
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 tickers = ['GOOG', 'CSCO', 'LOGI', 'AMZN', 'AAPL']
7 start_date = '2015-01-01'
8
9 data = yf.download(tickers, start=start_date, auto_adjust=True)['Close']
10
11 monthly_prices = data.resample('BM').last()
12 monthly_returns = monthly_prices.pct_change().dropna()
13
14 print("First 5 rows of monthly returns:\n", monthly_returns.head())
15
16 # Compute the vector of sample means
17 sample_means = monthly_returns.mean()
18 print("\nSample Means of Monthly Returns:\n", sample_means)
19
20 # Compute the matrix of sample variances and covariances
21 sample_cov_matrix = monthly_returns.cov()
22 print("\nSample Variance-Covariance Matrix of Monthly Returns:\n",
      sample_cov_matrix)
23
24 # Normalize prices (base = 100 at the start date)
25 normalized_prices = (monthly_prices / monthly_prices.iloc[0]) * 100
26
27 # Plot the normalized price chart
28 plt.figure(figsize=(10, 6))
29 for ticker in tickers:
30     plt.plot(normalized_prices.index, normalized_prices[ticker], label=ticker)
31
32 plt.title('Normalized Monthly Price Chart (Base 100)')
33 plt.xlabel('Date')
34 plt.ylabel('Normalized Price')
35 plt.legend()
36 plt.grid(True)
37 plt.tight_layout()
38
39 # Mean returns and variances-covariances of the assets
40 num_assets = len(tickers)
41 mean_returns = sample_means.values
42 cov_matrix = sample_cov_matrix.values
43 inv_cov_matrix = np.linalg.inv(cov_matrix)
44 ones = np.ones(num_assets)
45
46 # Global Minimum Variance (GMV) Portfolio
47 w_gmv = (inv_cov_matrix @ ones) / (ones.T @ inv_cov_matrix @ ones)
48 gmv_mean = w_gmv.T @ mean_returns
49 gmv_var = w_gmv.T @ cov_matrix @ w_gmv
50 gmv_stdev = np.sqrt(gmv_var)

```

```

51
52 print("\n--- Global Minimum Variance Portfolio ---")
53 for i, ticker in enumerate(tickers):
54     print(f"Weight {ticker}: {w_gmv[i]:.4f}")
55 print(f"Expected Return: {gmv_mean:.6f}")
56 print(f"Standard Deviation: {gmv_stdev:.6f}")
57
58 # Tangency Portfolio
59 rf = 0.00025
60 excess_returns = mean_returns - rf
61 w_tan = (inv_cov_matrix @ excess_returns) / (ones.T @ inv_cov_matrix @
        excess_returns)
62 tan_mean = w_tan.T @ mean_returns
63 tan_var = w_tan.T @ cov_matrix @ w_tan
64 tan_stdev = np.sqrt(tan_var)
65
66 print("\n--- Tangency Portfolio ---")
67 for i, ticker in enumerate(tickers):
68     print(f"Weight {ticker}: {w_tan[i]:.4f}")
69 print(f"Expected Return: {tan_mean:.6f}")
70 print(f"Standard Deviation: {tan_stdev:.6f}")
71
72 # Efficient frontier (combinations of GMV and Tangency)
73 weights = np.linspace(-1.0, 2.5, 200)
74 port_returns = []
75 port_stdevs = []
76 for w in weights:
77     w_port = w * w_tan + (1 - w) * w_gmv
78     port_returns.append(w_port.T @ mean_returns)
79     port_stdevs.append(np.sqrt(w_port.T @ cov_matrix @ w_port))
80
81 port_returns = np.array(port_returns)
82 port_stdevs = np.array(port_stdevs)
83
84 # Capital Allocation Line (CAL)
85 cal_stdevs = np.linspace(0, max(port_stdevs)*1.1, 100)
86 sharpe_ratio = (tan_mean - rf) / tan_stdev
87 cal_returns = rf + sharpe_ratio * cal_stdevs
88
89 # Plot Efficient Frontier
90 plt.figure(figsize=(10, 6))
91 plt.plot(port_stdevs, port_returns, label='Efficient Frontier', color='blue')
92 plt.scatter(gmv_stdev, gmv_mean, color='red', marker='o', s=100, label='GMV
        Portfolio', zorder=5)
93 plt.scatter(tan_stdev, tan_mean, color='green', marker='*', s=150, label='Tangency
        Portfolio', zorder=5)
94 plt.plot(cal_stdevs, cal_returns, label='Capital Allocation Line (CAL)',
        color='orange', linestyle='--')
95
96 # Plot the individual assets
97 for i, ticker in enumerate(tickers):
98     std = np.sqrt(cov_matrix[i, i])

```

```

99     mean = mean_returns[i]
100     plt.scatter(std, mean, color='black', marker='x', s=100, zorder=5)
101     plt.annotate(ticker, (std, mean), textcoords="offset points", xytext=(0,10),
        ha='center')
102
103 plt.title('Efficient Frontier and Portfolio Analysis')
104 plt.xlabel('Standard Deviation (Risk)')
105 plt.ylabel('Expected Return')
106 plt.legend()
107 plt.grid(True)
108 plt.tight_layout()
109 plt.show()

```

A.5. R scripts for question 3.2: Industry momentum

Listing 5: Code for matrix extraction. I took a code I already used before for Fama-French portfolios where time series were in daily or monthly frequencies.

```

1  library(tidyverse)
2  library(stringr)
3  library(dplyr)
4
5
6  all_folders <- c(
7    "30_Industry_Portfolios_CSV",
8    "F-F_Research_Data_Factors_CSV"
9  )
10
11 process_french_data <- function(folder_name) {
12
13   # Construct input and output file paths dynamically
14   file_name <- str_replace(folder_name, "(?i)_csv$", ".csv")
15   input_file_path <- file.path("raw data", folder_name, file_name)
16   output_file_path <- file.path("input data", file_name)
17
18   if (!file.exists(input_file_path)) {
19     warning(paste("File not found, skipping:", input_file_path))
20     return(NULL)
21   }
22
23   # Set date boundaries and regular expressions based on sampling frequency
24   is_daily <- str_detect(tolower(folder_name), "daily")
25
26   if (is_daily) {
27     start_date <- 19700101
28     end_date <- 20251231
29     date_regex <- "^\\d{8}$"
30   } else {
31     start_date <- 192707
32     end_date <- 202603
33     date_regex <- "^\\d{6}$"

```

```

34 }
35
36 is_factor_file <- str_detect(folder_name, "Factors")
37
38 # Load the raw text lines from the data file
39 all_lines <- readLines(input_file_path, warn = FALSE)
40
41 if (is_factor_file) {
42
43   # Identify the header line using character matching for factor labels
44   header_idx <- which(
45     str_detect(all_lines, "Mkt-RF") &
46     str_detect(all_lines, "SMB") &
47     str_detect(all_lines, "HML")
48   )
49   if (length(header_idx) == 0) {
50     warning(paste("Anchor not found in:", file_name, "- Skipping."))
51     return(NULL)
52   }
53   anchor_idx <- header_idx[1]
54   start_idx <- anchor_idx
55
56 } else {
57   # Identify the target table segment label for industry structures
58   anchor_string <- if (is_daily) {
59     "Average Value Weighted Returns -- Daily"
60   } else {
61     "Average Value Weighted Returns -- Monthly"
62   }
63
64   anchor_idx <- which(str_detect(all_lines, fixed(anchor_string)))
65   if (length(anchor_idx) == 0) {
66     warning(paste("Anchor not found in:", file_name, "- Skipping."))
67     return(NULL)
68   }
69   anchor_idx <- anchor_idx[1]
70   start_idx <- anchor_idx + 1
71 }
72
73 # Locate the terminal blank line separating the sub-table matrices
74 is_blank <- str_trim(all_lines) == ""
75 blank_lines_after_anchor <- which(is_blank & seq_along(all_lines) > start_idx)
76
77 if (length(blank_lines_after_anchor) == 0) {
78   end_idx <- length(all_lines)
79 } else {
80   end_idx <- min(blank_lines_after_anchor) - 1
81 }
82
83 # Slice the raw text to encapsulate the specific dataset window
84 table_lines <- all_lines[start_idx:end_idx]
85

```

```

86 # Parse the subset lines and reformat values to decimals
87 df <- read_csv(paste(table_lines, collapse = "\n"), show_col_types = FALSE)
88
89 clean_df <- df %>%
90   rename(date = 1) %>%
91   mutate(date = str_trim(as.character(date))) %>%
92   filter(str_detect(date, date_regex)) %>%
93   mutate(date = as.integer(date)) %>%
94   filter(date >= start_date & date <= end_date) %>%
95   mutate(across(-date, ~ {
96     val <- as.numeric(.)
97     val[val == -99.99 | val == -999] <- 0
98     return(val / 100)
99   }))
100
101 # Create the export directory if missing and write out the clean dataset
102 dir.create(dirname(output_file_path), showWarnings = FALSE, recursive = TRUE)
103 write_csv(clean_df, output_file_path)
104
105 cat("Successfully processed and saved:", file_name, "\n")
106 }
107
108 # Run the batch transformation routine over target directories
109 walk(all_folders, process_french_data)
110
111 # Import the formatted CSV records
112 X30_Industry_Portfolios <- read_csv("input data/30_Industry_Portfolios.csv")
113 FF_Research_Data_Factors <- read_csv("input data/F-F_Research_Data_Factors.csv")
114
115 # Compute portfolio returns relative to the risk-free asset rate
116 calculate_excess <- function(portfolio_df, factors_df) {
117
118   # Isolate the risk-free rate vector from the asset factor structure
119   rf_data <- factors_df %>%
120     dplyr::select(date, matches("(?i)^rf$")) %>%
121     dplyr::rename(rf = 2)
122
123   # Transform columns by mapping matches and subtracting the risk-free benchmarks
124   portfolio_df %>%
125     left_join(rf_data, by = "date") %>%
126     mutate(across(-c(date, rf), ~ . - rf)) %>%
127     select(-rf)
128 }
129
130 # Construct the excess asset matrix structure
131 X30_monthly_r_excess <- calculate_excess(X30_Industry_Portfolios,
132   FF_Research_Data_Factors)
133
134 rm(calculate_excess)

```

Listing 6: Code for the industry momentum strategy analysis.

```

1
2 source("code/Q3_2Stocks.R")
3
4 library(zoo)
5
6 # Sort target underlying datasets sequentially by date records
7 portfolio_data <- X30_Industry_Portfolios %>% arrange(date)
8 factors_data <- FF_Research_Data_Factors %>% arrange(date)
9
10 # Isolate risk-free asset return rates from the macro factor table
11 rf_data <- factors_data %>%
12   select(date, matches("(?i)^rf$")) %>%
13   rename(rf = 2)
14
15 # Calculate asset net excess returns over risk-free base rates
16 industry_returns <- portfolio_data %>%
17   left_join(rf_data, by = "date") %>%
18   mutate(across(-c(date, rf), ~ . - rf)) %>%
19   select(-rf) %>%
20   filter(date >= 192607)
21
22 # Average historical returns across a rolling twelve month backward window
23 rolling_mom <- industry_returns %>%
24   mutate(across(-date, ~ rollmean(lag(.), k = 12, fill = NA, align = "right"))) %>%
25   filter(date >= 192707)
26
27 # Reshape the output data matrix to long form for ranking operations
28 mom_long <- rolling_mom %>%
29   pivot_longer(cols = -date, names_to = "Industry", values_to = "Past_12M_Return")
30   %>%
31   filter(!is.na(Past_12M_Return))
32
33 # Assign sorting ranks within each independent monthly snapshot
34 mom_ranked <- mom_long %>%
35   group_by(date) %>%
36   mutate(Rank = rank(Past_12M_Return, ties.method = "first")) %>%
37   ungroup()
38
39 # Summarize the lifetime average rank locations across industry categories
40 industry_rank_summary <- mom_ranked %>%
41   group_by(Industry) %>%
42   summarize(Avg_Rank = mean(Rank)) %>%
43   arrange(Avg_Rank)
44
45 lowest_industry <- slice_head(industry_rank_summary, n = 1)
46 highest_industry <- slice_tail(industry_rank_summary, n = 1)
47
48 cat("Industry rank standings\n")
49 print(paste("Lowest average rank industry:", lowest_industry$Industry, "with an
50   average rank of:", round(lowest_industry$Avg_Rank, 2)))
51 print(paste("Highest average rank industry:", highest_industry$Industry, "with an
52   average rank of:", round(highest_industry$Avg_Rank, 2)))

```

```

50
51 # Measure the absolute chronological differences in industry ranks between months
52 turnover_analysis <- mom_ranked %>%
53   group_by(Industry) %>%
54   arrange(date) %>%
55   mutate(Rank_Change = abs(Rank - lag(Rank))) %>%
56   ungroup() %>%
57   filter(!is.na(Rank_Change))
58
59 avg_monthly_rank_shift <- mean(turnover_analysis$Rank_Change)
60 print(paste("Average monthly rank position shift per industry:",
61   round(avg_monthly_rank_shift, 2)))
62
63 # Map date fields to standard datetime formatting profiles for plotting
64 mom_ranked_plot_df <- mom_ranked %>%
65   mutate(Date_Parsed = as.Date(paste0(date, "01"), format="%Y%m%d"))
66
67 target_industry <- "Games"
68 ggplot(filter(mom_ranked_plot_df, Industry == target_industry), aes(x =
69   Date_Parsed, y = Rank)) +
70   geom_line(color = "blue", alpha = 0.6) +
71   scale_y_continuous(breaks = seq(1, 30, by = 5)) +
72   labs(
73     title = paste("Rank evolution of", target_industry, "industry"),
74     subtitle = "Monthly rank positions (1 = Worst 12m return, 30 = Best 12m
75       return)",
76     x = "Year",
77     y = "Rank position"
78   ) +
79   theme_minimal(base_size = 12)
80
81 ggplot(industry_rank_summary, aes(x = reorder(Industry, Avg_Rank), y = Avg_Rank)) +
82   geom_bar(stat = "identity", fill = "steelblue") +
83   coord_flip() +
84   labs(
85     title = "Average rank across all 30 industries, based on past average returns",
86     x = "Industry",
87     y = "Average historical rank"
88   ) +
89   theme_minimal()
90
91 # Reshape asset historical returns to facilitate data join operations
92 industry_returns_long <- industry_returns %>%
93   pivot_longer(cols = -date, names_to = "Industry", values_to = "Excess_Return")
94
95 # Merge dynamic factor ranks with contemporaneous performance data
96 portfolio_mapping <- mom_ranked %>%
97   left_join(industry_returns_long, by = c("date", "Industry"))
98
99 # Classify industries into discrete winner and loser equal-weighted buckets
100 strategy_returns <- portfolio_mapping %>%

```

```

99 mutate(Portfolio_Type = if_else(Rank > 15, "Winner", "Loser")) %>%
100 group_by(date, Portfolio_Type) %>%
101 summarize(Port_Excess_Return = mean(Excess_Return, na.rm = TRUE), .groups =
    "drop") %>%
102 pivot_wider(names_from = Portfolio_Type, values_from = Port_Excess_Return)
103
104 # Pull down historical benchmark equity premium returns
105 market_benchmark <- FF_Research_Data_Factors %>%
106   arrange(date) %>%
107   filter(date >= 192707) %>%
108   select(date, 'Mkt-RF')
109
110 # Construct a single reference table holding all structural returns
111 final_returns_df <- strategy_returns %>%
112   left_join(market_benchmark, by = "date")
113
114 # Compute mean, deviation, and Sharpe statistics across strategy vectors
115 performance_summary <- final_returns_df %>%
116   summarize(
117     Winner_Avg_Excess = mean(Winner),
118     Winner_Std_Dev = sd(Winner),
119     Winner_Monthly_SR = Winner_Avg_Excess / Winner_Std_Dev,
120     Winner_Annualized_SR = Winner_Monthly_SR * sqrt(12),
121
122     Loser_Avg_Excess = mean(Loser),
123     Loser_Std_Dev = sd(Loser),
124     Loser_Monthly_SR = Loser_Avg_Excess / Loser_Std_Dev,
125     Loser_Annualized_SR = Loser_Monthly_SR * sqrt(12),
126
127     Market_Avg_Excess = mean('Mkt-RF'),
128     Market_Std_Dev = sd('Mkt-RF'),
129     Market_Monthly_SR = Market_Avg_Excess / Market_Std_Dev,
130     Market_Annualized_SR = Market_Monthly_SR * sqrt(12)
131   )
132
133 cat("\nWinner portfolio performance metrics\n")
134 cat(paste("Avg monthly excess return: ",
135   round(performance_summary$Winner_Avg_Excess, 4), "\n"))
136 cat(paste("Monthly standard deviation: ", round(performance_summary$Winner_Std_Dev,
137   4), "\n"))
138 cat(paste("Monthly sharpe ratio: ", round(performance_summary$Winner_Monthly_SR,
139   4), "\n"))
140 cat(paste("Annualized sharpe ratio: ",
141   round(performance_summary$Winner_Annualized_SR, 4), "\n\n"))
142
143 cat("\nLoser portfolio performance metrics\n")
144 cat(paste("Avg monthly excess return: ",
145   round(performance_summary$Loser_Avg_Excess, 4), "\n"))
146 cat(paste("Monthly standard deviation: ", round(performance_summary$Loser_Std_Dev,
147   4), "\n"))
148 cat(paste("Monthly sharpe ratio: ", round(performance_summary$Loser_Monthly_SR,
149   4), "\n"))

```

```

143 cat(paste("Annualized sharpe ratio: ",
            round(performance_summary$Loser_Annualized_SR, 4), "\n\n"))
144
145 cat("Market index benchmark\n")
146 cat(paste("Annualized sharpe ratio: ",
            round(performance_summary$Market_Annualized_SR, 4), "\n"))
147
148
149 # Reintegrate risk-free rates to build total nominal returns
150 rf_reconstruct <- FF_Research_Data_Factors %>%
151   arrange(date) %>%
152   filter(date >= 192707) %>%
153   select(date, RF)
154
155 total_returns_df <- final_returns_df %>%
156   left_join(rf_reconstruct, by = "date") %>%
157   mutate(
158     Winner_Total = Winner + RF,
159     Loser_Total = Loser + RF,
160     Market_Total = 'Mkt-RF' + RF,
161     Ind_Mom_Total = Winner_Total - Loser_Total
162   )
163
164 # Calculate compound product curves tracking growth from base investment
165 cumulative_df <- total_returns_df %>%
166   arrange(date) %>%
167   mutate(
168     Cum_Winner = cumprod(1 + Winner_Total),
169     Cum_Loser = cumprod(1 + Loser_Total),
170     Cum_Market = cumprod(1 + Market_Total),
171     Cum_Ind_Mom = cumprod(1 + Ind_Mom_Total),
172     Date_Parsed = as.Date(paste0(date, "01"), format="%Y%m%d")
173   )
174
175 # Transform structural accumulation vectors into tidy plotting shapes
176 cumulative_long <- cumulative_df %>%
177   select(Date_Parsed, Cum_Winner, Cum_Loser, Cum_Market, Cum_Ind_Mom) %>%
178   pivot_longer(cols = -Date_Parsed, names_to = "Portfolio", values_to =
179     "Cumulative_Value") %>%
180   mutate(Portfolio = case_when(
181     Portfolio == "Cum_Winner" ~ "Winner Portfolio",
182     Portfolio == "Cum_Loser" ~ "Loser Portfolio",
183     Portfolio == "Cum_Market" ~ "Market Index",
184     Portfolio == "Cum_Ind_Mom" ~ "Long/Short Ind-Mom"
185   ))
186
187 ggplot(cumulative_long, aes(x = Date_Parsed, y = Cumulative_Value, color =
188   Portfolio)) +
189   geom_line(size = 0.8) +
190   scale_y_log10(labels = scales::dollar_format()) +
191   labs(
192     title = "Cumulative returns of $1 invested",

```

```

191     x = "Year",
192     y = "Log cumulative wealth ($)",
193     color = "Strategy"
194 ) +
195 theme_minimal(base_size = 12) +
196 theme(legend.position = "bottom")
197
198 # Filter timelines to examine strategy drawdowns during the 2009 shift
199 crash_2009 <- total_returns_df %>%
200   filter(str_detect(as.character(date), "^2009")) %>%
201   select(date, Ind_Mom_Total, Market_Total, Winner_Total, Loser_Total)
202
203 cat("\nIndustry momentum performance in 2009\n")
204 print(crash_2009)
205
206 # Isolate the worst standalone performance periods across history
207 worst_months <- total_returns_df %>%
208   select(date, Ind_Mom_Total, Market_Total, Winner_Total, Loser_Total) %>%
209   arrange(Ind_Mom_Total) %>%
210   slice_head(n = 10)
211
212 cat("\nTop 10 worst months for l/s ind-mom\n")
213 print(worst_months)

```

A.6. R script for question 4: Expectation hypothesis

Listing 7: Code for question 4: 2 tests on expectation hypothesis

```

1
2
3 library(readxl)
4 library(dplyr)
5 library(lmtest)
6 library(sandwich)
7
8 zcb_raw <- read_excel("raw data/LW_monthly.xlsx", col_names = FALSE, col_types =
9   "text")
10
11 # Locate the row containing the text "1 m"
12 anchor_idx <- which(apply(zcb_raw, 1, function(row) any(grepl("1 m", row))))[1]
13
14 # Convert the anchor row to character headers and assign "date" to the first element
15 headers <- as.character(zcb_raw[anchor_idx, ])
16 headers[1] <- "date"
17
18 # Subset everything below the anchor row and apply the extracted headers
19 zcb <- zcb_raw[(anchor_idx + 1):nrow(zcb_raw), ]
20 colnames(zcb) <- headers
21
22 # Filter out empty rows based on the date column and cast all variables to numeric
23 zcb <- zcb %>%

```

```

23   filter(!is.na(date)) %>%
24   mutate(across(everything(), as.numeric))
25
26
27   # Set up regression and bootstrap configurations
28   n_values <- c(2, 3, 4, 6, 9, 12, 24, 36, 48)
29   m_values <- c(1, 2, 3, 4, 6, 12)
30   B <- 1000
31   block_size <- 12
32
33   results <- data.frame()
34
35   for (n in n_values) {
36     for (m in m_values) {
37       if (n <= m) next
38
39       # Calculate the remaining maturity after m periods
40       rem_mat <- n - m
41
42       # Skip iteration if maturity values exceed data dimensions
43       if (n > ncol(zcb) | m > ncol(zcb) | rem_mat > ncol(zcb)) next
44
45       # Pull out yield series as numeric vectors using element selection
46       y_n <- as.numeric(zcb[[n]])
47       y_m <- as.numeric(zcb[[m]])
48       y_rem <- as.numeric(zcb[[rem_mat]])
49
50       # Construct the dependent variable representing long yield changes
51       dep_var <- as.numeric(dplyr::lead(y_rem, m) - y_n)
52
53       # Construct the independent variable representing the scaled yield spread
54       indep_var <- as.numeric((m / rem_mat) * (y_n - y_m))
55
56       # Pack variables into a dataframe and drop terminal rows containing missing
57       # lead elements
58       data_reg <- data.frame(Y = dep_var, X = indep_var)
59       data_reg <- na.omit(data_reg)
60
61       if (nrow(data_reg) == 0) next
62
63       # Estimate the primary baseline OLS model
64       fit <- lm(Y ~ X, data = data_reg)
65       alpha1_hat <- coef(fit)["X"]
66
67       # Compute Newey-West standard errors adjusted for moving average terms
68       nw_vcov <- NeweyWest(fit, lag = m - 1, prewhite = FALSE, adjust = TRUE)
69       se_alpha1_hat <- sqrt(diag(nw_vcov))["X"]
70
71       # Calculate the empirical t-statistic testing the null hypothesis of equality
72       # to 1
73       t_stat_hat <- (alpha1_hat - 1) / se_alpha1_hat

```

```

73 # Initialize containers for the moving block bootstrap distributions
74 n_obs <- nrow(data_reg)
75 alpha1_boot <- numeric(B)
76 t_boot <- numeric(B)
77
78 # Enforce reproducibility for the randomized bootstrap resampling loop
79 set.seed(123)
80 for (b in 1:B) {
81   # Resample random historical blocks with replacement
82   start_indices <- sample(1:(n_obs - block_size + 1), ceiling(n_obs /
83     block_size), replace = TRUE)
84   boot_idx <- unlist(lapply(start_indices, function(i) i:(i + block_size - 1)))
85   boot_idx <- boot_idx[1:n_obs]
86
87   boot_data <- data_reg[boot_idx, ]
88
89   # Estimate the OLS parameters on the bootstrapped data block
90   boot_fit <- lm(Y ~ X, data = boot_data)
91   a1_b <- coef(boot_fit)["X"]
92
93   # Extract bootstrap standard errors while catching potential matrix
94   # singularities
95   nw_vcov_b <- tryCatch({
96     NeweyWest(boot_fit, lag = m - 1, prewhite = FALSE, adjust = TRUE)
97   }, error = function(e) matrix(NA, 2, 2))
98
99   se_a1_b <- sqrt(diag(nw_vcov_b))["X"]
100
101   alpha1_boot[b] <- a1_b
102
103   # Shift the bootstrap test statistics to center around the empirical point
104   # estimate
105   t_boot[b] <- (a1_b - alpha1_hat) / se_a1_b
106 }
107
108 # Remove records reflecting failed bootstrap iterations
109 valid_boot <- !is.na(t_boot) & !is.na(alpha1_boot)
110 alpha1_boot <- alpha1_boot[valid_boot]
111 t_boot <- t_boot[valid_boot]
112
113 # Check the proportion of bootstrap estimates extending further away from unity
114 if (alpha1_hat < 1) {
115   frac_dir <- mean(alpha1_boot < alpha1_hat)
116 } else {
117   frac_dir <- mean(alpha1_boot > alpha1_hat)
118 }
119
120 # Check the proportion of bootstrap t-statistics exceeding the empirical
121 # boundary
122 frac_t <- mean(abs(t_boot) > abs(t_stat_hat))
123
124 # Append calculated indicators to the final collection dataframe

```

```

121     results <- rbind(results, data.frame(
122       n = n,
123       m = m,
124       alpha1_hat = alpha1_hat,
125       SE_alpha1 = se_alpha1_hat,
126       Frac_dir = frac_dir,
127       Frac_t = frac_t
128     ))
129   }
130 }
131
132 # Standardize output tables and map significance indicators
133 results$p_val <- 2 * (1 - pnorm(abs((results$alpha1_hat - 1) / results$SE_alpha1)))
134 results$stars <- ifelse(results$p_val < 0.01, "***",
135   ifelse(results$p_val < 0.05, "**",
136     ifelse(results$p_val < 0.1, "*", "")))
137
138 results$alpha1_hat <- round(results$alpha1_hat, 3)
139 results$alpha1_str <- paste0(results$alpha1_hat, results$stars)
140 results$SE_alpha1 <- round(results$SE_alpha1, 4)
141 results$Frac_dir <- round(results$Frac_dir, 3)
142 results$Frac_t <- round(results$Frac_t, 3)
143
144 final_table <- results[, c("n", "m", "alpha1_str", "SE_alpha1", "Frac_dir",
145   "Frac_t")]
146 colnames(final_table) <- c("n", "m", "alpha1_hat", "SE(alpha1_hat)", "Frac_dir",
147   "Frac_t")
148
149 cat("\n--- Test 1 Results: Long Yield Changes Over Life of Short Bond ---\n")
150 print(final_table, row.names = FALSE)
151
152 # Reset structural data collection for the second experiment layout
153 results_t2 <- data.frame()
154
155 for (n in n_values) {
156   for (m in m_values) {
157     # Isolate sample matches mapping to integer multiple maturities
158     if (n <= m || n %% m != 0) next
159
160     if (n > ncol(zcb) | m > ncol(zcb)) next
161
162     k <- n / m
163
164     y_n <- as.numeric(zcb[[n]])
165     y_m <- as.numeric(zcb[[m]])
166
167     # Accumulate leading values to evaluate perfect-foresight short yield paths
168     future_sum <- y_m
169     for (i in 1:(k - 1)) {
170       future_sum <- future_sum + as.numeric(dplyr::lead(zcb[[m]], i * m))
171     }
172   }
173 }

```



```

171 dep_var <- (future_sum / k) - y_m
172
173 # Measure the baseline current spread structure
174 indep_var <- y_n - y_m
175
176 data_reg <- data.frame(Y = dep_var, X = indep_var)
177 data_reg <- na.omit(data_reg)
178
179 if (nrow(data_reg) == 0) next
180
181 # Fit the second baseline OLS structure
182 fit <- lm(Y ~ X, data = data_reg)
183 beta1_hat <- coef(fit)["X"]
184
185 # Estimate long-horizon Newey-West standard errors to clean overlap biases
186 nw_vcov <- NeweyWest(fit, lag = n - m, prewhite = FALSE, adjust = TRUE)
187 se_beta1_hat <- sqrt(diag(nw_vcov))["X"]
188
189 t_stat_hat <- (beta1_hat - 1) / se_beta1_hat
190
191 n_obs <- nrow(data_reg)
192 beta1_boot <- numeric(B)
193 t_boot <- numeric(B)
194
195 set.seed(123)
196 for (b in 1:B) {
197   start_indices <- sample(1:(n_obs - block_size + 1), ceiling(n_obs /
198     block_size), replace = TRUE)
199   boot_idx <- unlist(lapply(start_indices, function(i) i:(i + block_size - 1)))
200   boot_idx <- boot_idx[1:n_obs]
201
202   boot_data <- data_reg[boot_idx, ]
203
204   boot_fit <- lm(Y ~ X, data = boot_data)
205   b1_b <- coef(boot_fit)["X"]
206
207   nw_vcov_b <- tryCatch({
208     NeweyWest(boot_fit, lag = n - m, prewhite = FALSE, adjust = TRUE)
209   }, error = function(e) matrix(NA, 2, 2))
210
211   se_b1_b <- sqrt(diag(nw_vcov_b))["X"]
212
213   beta1_boot[b] <- b1_b
214   t_boot[b] <- (b1_b - beta1_hat) / se_b1_b
215 }
216
217 valid_boot <- !is.na(t_boot) & !is.na(beta1_boot)
218 beta1_boot <- beta1_boot[valid_boot]
219 t_boot <- t_boot[valid_boot]
220
221 if (beta1_hat < 1) {
222   frac_dir <- mean(beta1_boot < beta1_hat)

```

```

222     } else {
223       frac_dir <- mean(beta1_boot > beta1_hat)
224     }
225
226     frac_t <- mean(abs(t_boot) > abs(t_stat_hat))
227
228     results_t2 <- rbind(results_t2, data.frame(
229       n = n,
230       m = m,
231       beta1_hat = beta1_hat,
232       SE_beta1 = se_beta1_hat,
233       Frac_dir = frac_dir,
234       Frac_t = frac_t
235     ))
236   }
237 }
238
239 # Clean and report summary analytics for the second testing framework
240 results_t2$p_val <- 2 * (1 - pnorm(abs((results_t2$beta1_hat - 1) /
241   results_t2$SE_beta1)))
242 results_t2$stars <- ifelse(results_t2$p_val < 0.01, "***",
243   ifelse(results_t2$p_val < 0.05, "**",
244     ifelse(results_t2$p_val < 0.1, "*", "")))
245
246 results_t2$beta1_hat <- round(results_t2$beta1_hat, 3)
247 results_t2$beta1_str <- paste0(results_t2$beta1_hat, results_t2$stars)
248 results_t2$SE_beta1 <- round(results_t2$SE_beta1, 4)
249 results_t2$Frac_dir <- round(results_t2$Frac_dir, 3)
250 results_t2$Frac_t <- round(results_t2$Frac_t, 3)
251
252 final_table_t2 <- results_t2[, c("n", "m", "beta1_str", "SE_beta1", "Frac_dir",
253   "Frac_t")]
254 colnames(final_table_t2) <- c("n", "m", "beta1_hat", "SE(beta1_hat)", "Frac_dir",
255   "Frac_t")
256
257 cat("\n--- Test 2 Results: Short Yield Changes Over Life of Long Bond ---\n")
258 print(final_table_t2, row.names = FALSE)

```
